

Zuga Technologies Design Bible

The master visual language inherited by every Zuga product.

Master accent: acid lime #a3e635

1. Mission & Principles

1. Mission & Principles

Authored: 2026-04-25 Status: Sealed (no Mike checkpoint required)

1.1 Mission Statement

Zuga Technologies builds the tooling layer for the next generation of human-AI collaboration — a fleet of purpose-built studios under one identity, compounding toward a future where autonomous agents handle the cognitive load so people can spend time on what actually matters. We ship fast, stay coherent, and earn trust by working visibly in public. The studio fleet is the near-horizon bet; the far-horizon bet is that the autonomous-agent architecture underpinning all of it is a real research category worth proving, not just a product line.

1.2 Design Principles

These are operating principles — concrete enough that a designer or engineer can apply them in code review. When you're unsure whether a decision is on-brand, run it against this list.

P1. Direct over decorative

Every visual or copy choice should earn its place by communicating something. Ornamentation that doesn't carry meaning is noise. A button label should say what happens; an illustration should clarify, not impress.

P2. Coherent system, specialized surfaces

Studios share a common foundation (tokens, type scale, spacing, master mark) but adapt at the surface level for their domain. A wellness screen should feel different from a trading screen — same blood, different posture. Don't enforce uniformity where context calls for distinction.

P3. Ship the thing, not the concept

Design for the version that ships in the next two weeks, not for a theoretical future state. Document aspirational states separately. Present-tense interfaces are built against present-tense constraints.

P4. Accessibility is not optional

WCAG AA contrast is the minimum bar, not a stretch goal. Every interactive element has a focus state. Every image has alt text. This isn't a compliance checkbox — users who depend on these features have no fallback.

P5. Hierarchy over entropy

One primary action per screen. One primary color per context. When everything competes for attention, nothing wins. Restraint in hierarchy is a feature, not a limitation.

P6. Terminology is architecture

The words we use for products, features, and concepts are load-bearing. Changing "studios" to "apps" mid-product is a refactor, not a copy edit. Names go in the voice-and-tone dictionary before they go in the UI.

P7. Earn trust incrementally

New patterns and visual directions need to prove themselves in one surface before they propagate. Speculative redesigns ship as experiments, not system-wide rewrites. Rollback paths are part of the design.

1.3 Anti-Patterns

These are explicit “we don’t do this” rules with rationale. If a proposal lands in this list, it needs a compelling override argument before we proceed.

No corporate-speak. Phrases like “leverage synergies,” “seamless experience,” or “world-class” are banned from copy. They’re signals that the writer doesn’t trust the work to speak for itself.

No dark patterns. Misleading CTA placement, hidden cancellation flows, countdown timers on non-expiring offers — these destroy the long-term trust that drives retention. Short-term conversion gains aren’t worth it.

No inconsistent token usage. Don’t reach for hex values or raw pixel counts when a design token exists. Undocumented one-offs accumulate into maintenance debt that eventually breaks the system.

No “just this once” accessibility shortcuts. Skipping focus states, using color alone to convey state, or leaving aria labels empty with “we’ll fix it later” is how permanently broken accessibility accumulates.

No brand inflation. Sub-brands don’t invent their own primary color, typeface, or logo treatment outside the Branded House contract (§2). Each studio inherits the master mark and gets one accent color. That’s the deal.

No terminology drift. “ZugaChat” and “Zugabot” are not interchangeable. “ZugaTokens” doesn’t become “credits” in user-facing copy. “Studios” doesn’t become “apps.” See §3 terminology dictionary. Drift in one surface sets a precedent everywhere.

No pattern sprawl. If a new component pattern is created for one studio, it either goes into the shared component library immediately or it gets deleted after the specific use case resolves. Studio-local patterns that persist become orphans.

1.4 Decision Framework

When sections of this bible conflict with each other, follow this hierarchy:

Accessibility > Brand Foundation > Mission > everything else.

In prose: if a brand decision would compromise WCAG AA, accessibility wins. If a mission framing contradicts a brand-foundation rule already approved by Mike, the brand-foundation rule wins. Mission frames *why*; the bible body specifies *how*.

This hierarchy exists because:

Accessibility failures have real users as victims and legal exposure as consequence.

Brand-foundation decisions (§2) are the ones Mike has hard-veto authority on — once approved, they’re stable contracts.

Mission and principles (this section) are directional, not prescriptive — they lose to specific approved decisions.

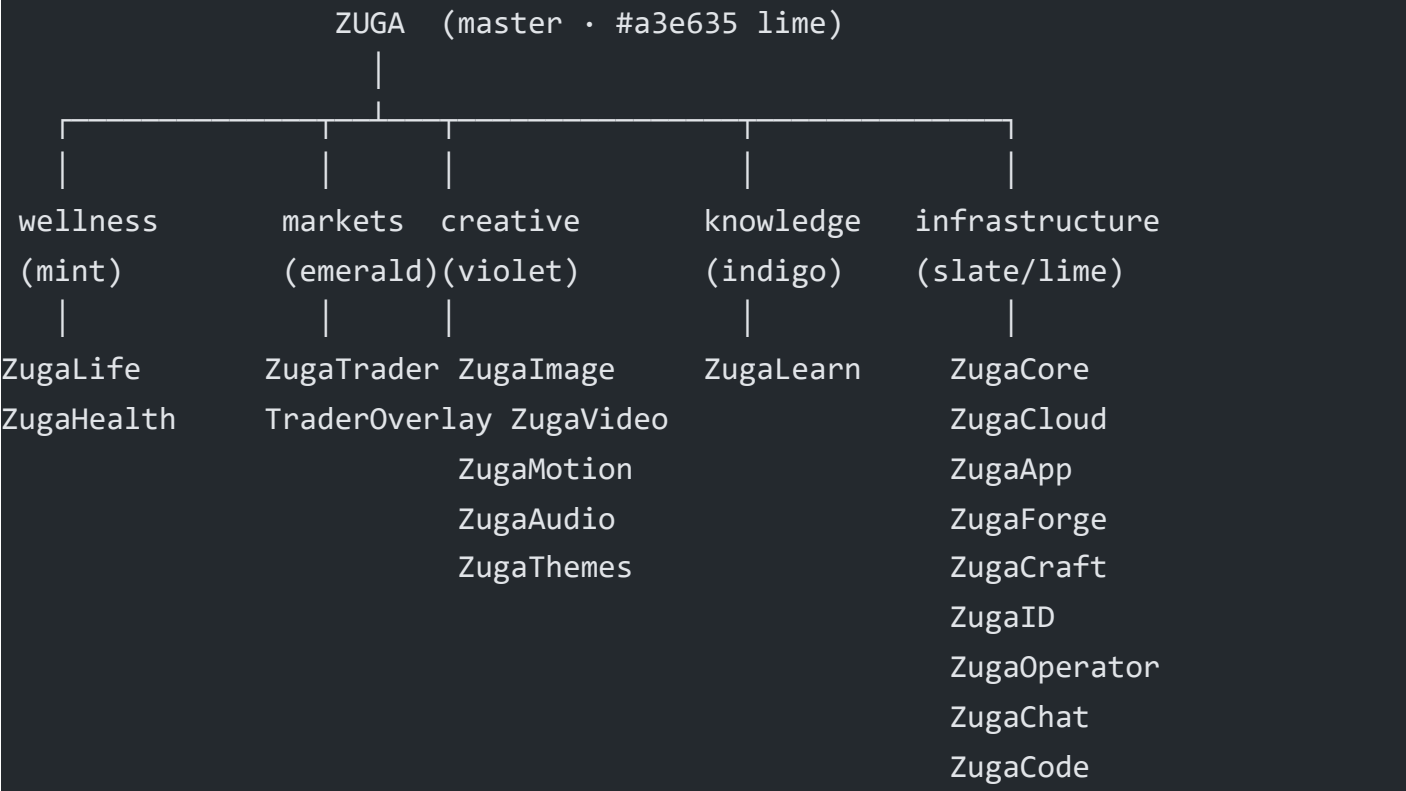
2. Brand Foundation

2. Brand Foundation

Authored: 2026-04-25 Mike checkpoints: wm01, wm02, wm03

2.1 Brand Architecture

Zuga Technologies operates as a **Branded House** — not a house of brands. Every product carries the Zuga prefix and inherits the master mark. Sub-brands get one accent color and adapt tone for their domain; they don’t build separate identities.



Sub-brands inherit: master wordmark, type system, spacing tokens. Sub-brands own: one accent hue, surface-level illustration style, domain-specific iconography.

2.2 Wordmark

The wordmark is the primary identity asset. It appears on every product surface, every public document, and every release artifact. The wordmark direction is [MIKE-CHECKPOINT-wm01].

● **MIKE-CHECKPOINT-wm01** — confirm wordmark direction

Field	Value
Antonio’s draft	All-caps “ZUGA” set in a geometric sans — Inter or equivalent, weight 700, tracked +20. The wordmark is set in master lime (#a3e635) on dark surfaces and near-black (#0f172a) on light surfaces. No tagline lockup at small sizes.
Alternates considered	Mixed-case “Zuga” (title case) — rejected: feels like a consumer startup, loses authority at product scale. Lowercase “zuga” — rejected: too casual for a company

Field	Value
	that runs financial tooling.
Renders on	Corp landing header, ZugaApp navbar, all studio top-bars, email headers, GitHub org avatar, social card OG images
✓ Approve	→ flip to [MIKE-APPROVED-wm01: YYYY-MM-DD] ✎ Modify → edit “Antonio’s draft” field

2.3 Logo Mark / Glyph

For contexts where the full wordmark doesn’t fit (app icons, favicons, avatar slots), Zuga uses a single-glyph mark. The glyph treatment is [MIKE-CHECKPOINT-wm02].

● MIKE-CHECKPOINT-wm02 — confirm logo mark / glyph treatment

Field	Value
Antonio’s draft	Geometric “Z” constructed from two horizontal strokes and one diagonal — all strokes equal weight, rounded terminals, no serifs. The Z sits inside a square container with 15% padding. Background: master lime (#a3e635). Foreground: near-black (#0a0a0a) — lime is a light-forward accent, so the glyph must sit in dark foreground, not white, to hold contrast. Corner radius: 22% of container width (feels like a modern app icon, not a logo stamp).
Alternates considered	Circular container — rejected: squares tile better in OS icon grids and favicon contexts. Stylized Z with a cutting diagonal notch — rejected: too aggressive, doesn’t read cleanly at 16px. Wordmark-only (no glyph) — rejected: doesn’t work in 32px favicon or 1:1 avatar slot.
Renders on	Browser favicon (16px, 32px), iOS/Android app icon (1024px source), GitHub org avatar, Discord server icon, ZugaApp PWA manifest icon
✓ Approve	→ flip to [MIKE-APPROVED-wm02: YYYY-MM-DD] ✎ Modify → edit “Antonio’s draft” field

2.4 Master Color Anchor

The Zuga master accent hue is the single most visible brand signal across all surfaces. The master hue is [MIKE-CHECKPOINT-wm03].

● MIKE-CHECKPOINT-wm03 — confirm master accent hue

Field	Value
Antonio’s draft	#a3e635 (Tailwind lime-400). Passes WCAG AAA against #0a0a0a (near-black surfaces) — measured contrast $\approx 13:1$, which clears the 7:1 AAA threshold for normal text with room to spare. Fails AA against white (#ffffff) — never used as a background color behind body text. Lime is a light-forward accent, unlike the old cyan-700 default: any foreground drawn on top of a lime fill must be dark (#0a0a0a), not white. Used for:

Field	Value
	primary CTAs, active nav indicators, focus rings, master wordmark on dark surfaces, hyperlinks in body copy.
Alternates considered	#84cc16 (lime-500) — rejected: too somber for a tech-forward brand, loses energy at small sizes. #bef264 (lime-300) — rejected: fails WCAG AA on white backgrounds, feels juvenile. #3b82f6 (blue-500) — rejected: indistinguishable from a dozen SaaS products; lime is the differentiation.
Renders on	Corp landing CTAs, ZugaApp primary buttons, ZugaApp navbar active states, ZugaThemes accent pills, focus ring system-wide, master wordmark on dark surfaces

☒ Approve → flip to [MIKE-APPROVED-wm03: YYYY-MM-DD]
  Modify → edit "Antonio's draft" field

2.5 Brand Voice Anchors

The full voice system lives in §3. These are the three anchors that apply at the brand level — the adjectives Mike can hold the whole system to:

Direct. We say what we mean in the fewest words that carry the meaning. If you can cut a word without losing anything, cut it.

Technical but not exclusive. We use precise vocabulary because our users are capable. We don't use jargon to signal status or exclude newcomers.

Grounded. We don't make claims we can't back up. We don't hype features that aren't shipped. We don't promise futures we haven't committed to building.

3. Voice & Tone

3. Voice & Tone

Authored: 2026-04-25 Mike checkpoints: voice01–voice11

Voice is who we are. Tone is how we sound in a given context. Voice stays constant; tone shifts by surface. All decisions below are tagged for Mike review.

3.1 Master Tone

The single adjective set that defines Zuga writing at every surface is [MIKE-CHECKPOINT-voice01].

● **MIKE-CHECKPOINT-voice01** — confirm master tone descriptor

Field	Value
Antonio’s draft	Direct, technical, occasionally dry — never corporate. We write like a founder explaining something to a smart friend, not like a product manager writing for a press release. Confidence comes from specificity, not from superlatives.
Alternates considered	“Warm and approachable” — rejected: sounds like a consumer wellness brand, not an agent-fleet platform. “Bold and visionary” — rejected: too much pitch-deck energy, doesn’t hold at error-state copy.
Renders on	All surfaces — this is the baseline every other tone shift modifies

✓ Approve → flip to [MIKE-APPROVED-voice01: YYYY-MM-DD] ✎ Modify → edit “Antonio’s draft” field

3.2 Tone by Surface

Marketing (Landing pages, social, press)

The marketing tone is [MIKE-CHECKPOINT-voice02].

● **MIKE-CHECKPOINT-voice02** — confirm marketing tone

Field	Value
Antonio’s draft	Energetic but earned. We lead with what the product does, not what we aspire to be. Headlines state a concrete outcome (“Your AI agent fleet, one login”). Body copy explains the mechanism. No countdown timers, no vague “AI-powered” claims, no social proof fabrication. We can be punchy; we can’t be hollow.
Alternates considered	Hype-forward (“The future of work is here”) — rejected: every SaaS company says this; it’s become noise. Dry/technical only — rejected: marketing is still a hook; we’re allowed to have energy at the top of the funnel.
Renders on	zuatechnologies.com landing, ZugaApp marketing pages, social card copy, GitHub org README

✓ Approve → flip to [MIKE-APPROVED-voice02: YYYY-MM-DD] ✎ Modify → edit “Antonio’s draft” field

Product UI (In-app copy, button labels, tooltips)

The product-UI tone is [MIKE-CHECKPOINT-voice03].

● **MIKE-CHECKPOINT-voice03** — confirm product-UI tone

Field	Value
Antonio's draft	Terse and action-oriented. Button labels use imperative verbs: "Run", "Connect", "Dismiss", not "Click here to run" or "Please connect your account." Tooltip copy answers one question only. Status messages report state without editorializing ("Syncing 3 files" not "We're working hard to sync your files!").
Alternates considered	Conversational ("Hey, let's connect your account!") — rejected: adds friction to task-completion flows; users in the UI want to act, not chat. Passive ("Files are being synced") — rejected: hides agency and feels slower.
Renders on	All studio UIs — ZugaApp nav, ZugaLife habit cards, ZugaTrader order panel, ZugaCode activity feed, ZugaGamer overlay

✅ Approve → flip to [MIKE-APPROVED-voice03: YYYY-MM-DD] ✎ Modify → edit "Antonio's draft" field

Error States (Validation errors, API failures, 4xx/5xx)

The error-state tone is [MIKE-CHECKPOINT-voice04].

● **MIKE-CHECKPOINT-voice04** — confirm error-state tone

Field	Value
Antonio's draft	Honest and specific, never apologetic theater. State what broke, state what the user can do next, and stop. "Couldn't reach the server. Check your connection and try again." — not "Oops! Something went wrong on our end. We're so sorry for the inconvenience!" Blame should go to the system, not the user; but we don't perform guilt either.
Alternates considered	Cute/emoji-heavy errors ("🙈 Oops, that didn't work!") — rejected: condescending when something is actually broken; erodes trust for users dealing with a genuine failure. Clinical/code-dump errors ("Error 503: upstream connect error") — rejected: not actionable for most users; raw codes belong in developer consoles, not UI copy.
Renders on	ZugaApp error banners, ZugaLife form validation, ZugaTrader order rejection messages, ZugaCode hook failure notifications, all 4xx/5xx page copy

✅ Approve → flip to [MIKE-APPROVED-voice04: YYYY-MM-DD] ✎ Modify → edit "Antonio's draft" field

Empty States (Zero-data, first-use, no results)

The empty-state tone is [MIKE-CHECKPOINT-voice05].

● **MIKE-CHECKPOINT-voice05** — confirm empty-state tone

Field	Value
Antonio's draft	Contextual and forward-leaning. Tell the user what this space is for and what action fills it. "No habits yet. Add your first one to start tracking." — not a decorative

Field	Value
	illustration with “Nothing to see here!” Empty states are the best moment to orient a new user; they should carry real information, not filler copy.
Alternates considered	Whimsical/illustrated (“Your canvas is blank — let’s paint something amazing!”) — rejected: doesn’t scale across business-facing studios like ZugaTrader. Pure blank (no copy at all) — rejected: disorienting, especially for first-time users; the empty state is a teaching moment.
Renders on	ZugaLife habits/streaks/goals, ZugaTrader watchlist, ZugaImage gallery, ZugaCode review queue, ZugaChat history
✓ Approve → flip to [MIKE-APPROVED-voice05: YYYY-MM-DD] ✎ Modify → edit “Antonio’s draft” field	
Onboarding (First-run, signup, feature discovery)	
The onboarding tone is [MIKE-CHECKPOINT-voice06].	

● **MIKE-CHECKPOINT-voice06** — confirm onboarding tone

Field	Value
Antonio’s draft	Confident and tutorial-paced. We assume the user is smart and skip condescension (“You’re doing great!”). We don’t assume they know our jargon — first use of “studios,” “ZugaTokens,” or “Zugabot” gets a one-sentence definition inline, not a glossary redirect. Steps are numbered; nothing is hidden behind “you’ll figure it out.” We want users to feel capable after onboarding, not grateful.
Alternates considered	Gamified congratulations at every step — rejected: feels patronizing after the second badge, cheapens actual milestones. No onboarding copy (jump straight to the UI) — rejected: works for power users but alienates the target segment joining via the Anthropic partnership cohort.
Renders on	ZugaApp signup flow, ZugaLife first-run habit setup, ZugaTrader account connection wizard, ZugaCode observer install prompt
✓ Approve → flip to [MIKE-APPROVED-voice06: YYYY-MM-DD] ✎ Modify → edit “Antonio’s draft” field	

3.3 Grammar Rules

Pronouns and Contractions

The pronoun and contraction rule is [MIKE-CHECKPOINT-voice07].

● **MIKE-CHECKPOINT-voice07** — confirm pronoun and contraction rules

Field	Value
Antonio’s draft	We use “you” for the user and “we” for Zuga. Contractions are allowed and preferred in UI copy and marketing — “you’re” not “you are,” “we’ll” not “we will.” Oxford comma is mandatory in all lists of three or more (“studios, agents, and tokens” not “studios, agents and tokens”). Sentence case for UI labels (“Add habit” not “Add

Field	Value
	Habit”), title case for section headers and navigation (“Voice & Tone” not “Voice and tone”).
Alternates considered	Third-person brand voice (“Zuga will sync your data”) — rejected: creates unnecessary distance, reads as legal boilerplate. No contractions (formal register throughout) — rejected: clashes with the direct/blunt brand voice; formal language implies bureaucracy.
Renders on	All surfaces — this is a universal grammar rule applied system-wide
	 Approve → flip to [MIKE-APPROVED-voice07: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field

3.4 Terminology Dictionary

These entries are binding. Drift from any entry requires a terminology change proposal, not a one-off exception.

Zugabot vs ZugaChat

The Zugabot vs ZugaChat naming rule is [MIKE-CHECKPOINT-voice08].

 **MIKE-CHECKPOINT-voice08** — confirm Zugabot vs ZugaChat terminology rule

Field	Value
Antonio’s draft	“Zugabot” refers to the autonomous agent core — the system that executes tasks, manages memory, and runs without real-time human input. “ZugaChat” refers to the user-facing chat surface where humans talk to the agent in real time. Never conflate them. In UI copy: “Chat with Zugabot via ZugaChat” is correct; “Open Zugabot” as a label for the chat window is wrong — that label implies you’re opening the agent itself, not the interface.
Alternates considered	Treating them as interchangeable (“Zugabot/ZugaChat”) — rejected: they are architecturally distinct; conflation causes user confusion and engineering miscommunication. Using “Zugabot” as the user-facing name everywhere — rejected: the agent name in user copy sounds robotic; ZugaChat is the human-facing product surface.
Renders on	ZugaApp nav labels, marketing copy, onboarding steps, API documentation, internal engineering specs
	 Approve → flip to [MIKE-APPROVED-voice08: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field

ZugaTokens vs credits

The ZugaTokens currency-naming rule is [MIKE-CHECKPOINT-voice09].

 **MIKE-CHECKPOINT-voice09** — confirm ZugaTokens vs credits terminology rule

Field	Value
Antonio's draft	The user-facing currency is always "ZugaTokens" — capital Z, capital T, one word. Never "credits," "coins," "points," or "tokens" (lowercase). In sentence-level copy: "You have 240 ZugaTokens remaining." The internal/developer name for the unit may be "token" (lowercase) in code and database schemas. The user never sees the lowercase form in the product UI.
Alternates considered	"Credits" — rejected: generic, used by dozens of competitors, gives users no brand recall; also implies consumption without investment. "Zuga Credits" — rejected: "credits" still carries the wrong connotation; ZugaTokens is already established in the codebase and user-facing copy.
Renders on	ZugaApp wallet UI, pricing pages, ZugaLife feature gates, ZugaTrader execution costs display, Zugalimage generation cost display, all billing-related copy
Studios vs apps	 Approve → flip to [MIKE-APPROVED-voice09: YYYY-MM-DD]  Modify → edit "Antonio's draft" field The studios vs apps terminology rule is [MIKE-CHECKPOINT-voice10].

 **MIKE-CHECKPOINT-voice10** — confirm studios vs apps terminology rule

Field	Value
Antonio's draft	The plug-in units inside ZugaApp are called "studios" in all user-facing copy. Never "apps," "modules," "plugins," or "tools" in user-facing surfaces. In the food-court metaphor: ZugaApp is the food court (host shell); each studio is a food truck (specialized vendor). The metaphor is internal framing — don't use "food truck" in user-facing copy. Externally: "ZugaLife is a studio inside ZugaApp." In navigation labels: "Studios" (not "Apps").
Alternates considered	"Apps" — rejected: implies standalone installation; studios are integrated; using "apps" creates the wrong mental model about scope and portability. "Modules" — rejected: sounds like a content-management system, implies optional add-ons rather than first-class products.
Renders on	ZugaApp Studios directory, onboarding copy, marketing pages, ZugaApp navbar ("Studios" section label), all product descriptions
Food court vs food truck (internal framing)	 Approve → flip to [MIKE-APPROVED-voice10: YYYY-MM-DD]  Modify → edit "Antonio's draft" field

Food court vs food truck (internal framing)

The food court / food truck framing (internal-only) is [MIKE-CHECKPOINT-voice11].

 **MIKE-CHECKPOINT-voice11** — confirm food court / food truck internal framing rule

Field	Value
Antonio's draft	Internal engineering and design vocabulary only. "Food court" = ZugaApp (the host shell that aggregates studios). "Food truck" = an individual studio that operates inside the food court. The framing clarifies integration ownership: if a studio (food truck) has a problem, fix the food truck; don't renovate the food court for one vendor. This metaphor governs architecture decisions — a studio-specific problem is not a ZugaApp-layer problem unless the integration layer itself is broken. Never appears in user-facing copy.
Alternates considered	"Container / service" — rejected: too generic, maps to Docker mental models in engineering but loses the integration-ownership nuance for design and product discussions. "Platform / plug-in" — rejected: "plug-in" implies optional and detachable; studios are first-class products in the Branded House, not optional extensions.
Renders on	Internal engineering specs, design system documentation, architecture diagrams, code review discussions — NOT user-facing surfaces
✓ Approve → flip to [MIKE-APPROVED-voice11: YYYY-MM-DD] ✎ Modify → edit "Antonio's draft" field	

3.5 Do / Don't Quick Reference

Do	Don't
"Run the agent"	"Leverage the AI-powered assistant"
"240 ZugaTokens remaining"	"240 credits remaining"
"Open ZugaChat" (the interface)	"Open Zugabot" (for the chat UI)
"Add a studio from the directory"	"Install an app from the marketplace"
"Couldn't reach the server. Try again."	"Oops! Something went wrong on our end!"
"You have 3 habits this week."	"You're crushing it! 3 habits tracked!"
Oxford comma: "studios, agents, and tokens"	"studios, agents and tokens"
Sentence case on UI labels: "Add habit"	Title case on UI labels: "Add Habit"

4. Color Foundations

4. Color Foundations

Authored: 2026-04-25 Mike checkpoints: none (Tier 1 primitives are Antonio-final)

Tier 1 color primitives are brand-agnostic raw values. They have no meaning by themselves — meaning is assigned in §5 Color in Practice. The 13 ramps below are the complete primitive inventory. Hue confirm checkpoints (h01–h09) live in §5 where the *role* is decided. Hex values here are copied directly from `packages/design-tokens/src/tokens.css` — the bible documents what is shipped, it does not re-decide.

Every ramp runs 11 stops: 50, 100, 200, 300, 400, 500, 600, 700, 800, 900, 950. Lower numbers = lighter; higher numbers = darker. In dark mode, semantic roles that map to these ramps flip to higher-numbered stops (surface roles) or lighter stops (text roles). See §4.14 for dark-mode swap mechanics.

4.1 Slate — cool neutral

Role: the backbone neutral. All surface, text, and border tokens are drawn from this ramp.

Stop	Hex
------	-----

50	#f8fafc
----	---------

100	#f1f5f9
-----	---------

200	#e2e8f0
-----	---------

300	#cbd5e1
-----	---------

400	#94a3b8
-----	---------

500	#64748b
-----	---------

600	#475569
-----	---------

700	#334155
-----	---------

800	#1e293b
-----	---------

900	#0f172a
-----	---------

950	#020617
-----	---------

Status: Antonio-final. No hue checkpoint — slate is the system neutral, not a brand color.

4.2 Lime — master Zuga accent

Role: **the master Zuga brand accent** (unified 2026-06-30; see ADR-0004). Lime is light-forward — the identity stop lime-400 (#a3e635) is ALSO the functional stop, always paired with a DARK foreground (#0a0a0a). A filled lime element with dark text clears WCAG AAA on both light and dark surfaces. Do NOT use lime-400 as text on a light surface; use lime-700 (#4d7c0f) for accent text on light. Renders on: corp landing + zugabot.ai product CTAs, wordmark on dark, primary buttons, focus rings, links.

Stop Hex

50 #f7fee7
100 #ecfccb
200 #d9f99d
300 #bef264
400 #a3e635
500 #84cc16
600 #65a30d
700 #4d7c0f
800 #3f6212
900 #365314
950 #1a2e05

Status: Antonio-final (master accent, 2026-06-30). Supersedes the former cyan master (§4.2.1; ADR-0002 → ADR-0004).

4.2.1 Cyan — feedback / info hue (former master)

Role: --feedback-info only. Cyan was the master Zuga accent until 2026-06-30, when the brand unified to lime to match the zugabot.ai product identity. The cyan ramp is retained as a Tier-1 primitive for the info/feedback state; it is no longer a brand accent. cyan-500 (#06b6d4) still must not be used as text on white (fails AA — a raw-ramp contrast fact independent of role).

Stop Hex

50 #ecfeff
100 #cffafe
200 #a5f3fc
300 #67e8f9
400 #22d3ee
500 #06b6d4
600 #0891b2
700 #0e7490
800 #155e75
900 #164e63
950 #083344

Status: retained primitive; feedback-info role Antonio-final.

4.3 Green — feedback.success

Role: success feedback only. Not used for brand or accent. Separate from emerald (which carries the markets convention).

Stop Hex

50 #f0fdf4
100 #dcfce7
200 #bbf7d0
300 #86efac
400 #4ade80
500 #22c55e
600 #16a34a
700 #15803d
800 #166534
900 #14532d
950 #052e16

Status: Antonio-final. No hue checkpoint.

4.4 Emerald — markets family accent

Role: ZugaTrader and ZugaTraderOverlay. Hue confirm in §5 (checkpoint h09). Visually distinct from green — emerald reads as “money/markets” where green reads as “success state.”

Stop Hex

50 #ecfdf5
100 #d1fae5
200 #a7f3d0
300 #6ee7b7
400 #34d399
500 #10b981
600 #059669
700 #047857
800 #065f46
900 #064e3b
950 #022c22

Status: hue family in spec; specific stop confirmed in §5 h09.

4.5 Amber — feedback.warn

Role: warning feedback only. Not used for brand or accent. Dark-mode warning surfaces use amber-950 as the subtle background; text reverts to --text-primary (slate-50) for contrast — see §5 contrast table note.

Stop Hex

50 #ffffbe
100 #fef3c7

Stop Hex

200 #fde68a
300 #fcd34d
400 #fbbf24
500 #f59e0b
600 #d97706
700 #b45309
800 #92400e
900 #78350f
950 #451a03

Status: Antonio-final. No hue checkpoint.

4.6 Red — feedback.danger / ZugaShield

Role: danger feedback and ZugaShield studio accent. Dual-use is intentional: security (ZugaShield) maps naturally to a danger-signal hue. Studio assignment confirmed in §5 acc19.

Stop Hex

50 #fef2f2
100 #fee2e2
200 #fecaca
300 #fca5a5
400 #f87171
500 #ef4444
600 #dc2626
700 #b91c1c
800 #991b1b
900 #7f1d1d
950 #450a0a

Status: Antonio-final. No hue checkpoint.

4.7 Mint — wellness family accent

Role: ZugaLife and ZugaHealth. Hue confirm in §5 (checkpoint h02). Mint (teal-leaning) reads as health/wellness, distinct from both cyan (brand) and green (success).

Stop Hex

50 #f0fdfa
100 #ccfbf1
200 #99f6e4
300 #5eead4
400 #2dd4bf

Stop Hex

500 #14b8a6
600 #0d9488
700 #0f766e
800 #115e59
900 #134e4a
950 #042f2e

Status: hue family in spec; specific stop confirmed in §5 h02.

4.8 Violet — creative family accent

Role: ZugaImage, ZugaVideo, ZugaMotion. Hue confirm in §5 (checkpoint h03). Violet conveys creativity and visual production without the “brand” read of cyan.

Stop Hex

50 #f5f3ff
100 #ede9fe
200 #ddd6fe
300 #c4b5fd
400 #a78bfa
500 #8b5cf6
600 #7c3aed
700 #6d28d9
800 #5b21b6
900 #4c1d95
950 #2e1065

Status: hue family in spec; specific stop confirmed in §5 h03.

4.9 Blue — ZugaCode

Role: ZugaCode studio accent. Hue confirm in §5 (checkpoint h04). Blue is the canonical developer-tools convention. Kept separate from sky (cloud) for legible sibling distinction.

Stop Hex

50 #eff6ff
100 #dbeafe
200 #bdfbfe
300 #93c5fd
400 #60a5fa
500 #3b82f6
600 #2563eb
700 #1d4ed8

Stop Hex

800 #1e40af

900 #1e3a8a

950 #172554

Status: hue family in spec; specific stop confirmed in §5 h04.

4.10 Sky — ZugaCloud

Role: ZugaCloud studio accent. Hue confirm in §5 (checkpoint h05). Sky reads as “cloud/infrastructure” — lighter and airier than blue, which reads as “code/IDE.”

Stop Hex

50 #f0f9ff

100 #e0f2fe

200 #bae6fd

300 #7dd3fc

400 #38bdf8

500 #0ea5e9

600 #0284c7

700 #0369a1

800 #075985

900 #0c4a6e

950 #082f49

Status: hue family in spec; specific stop confirmed in §5 h05.

4.11 Magenta — ZugaAudio

Role: ZugaAudio studio accent. Hue confirm in §5 (checkpoint h06). Token name is `color.magenta.*`; the underlying CSS values are sourced from Tailwind’s `fuchsia` ramp — this is noted in `tokens.css` and intentional.

Stop Hex

50 #fdf4ff

100 #fae8ff

200 #f5d0fe

300 #f0abfc

400 #e879f9

500 #d946ef

600 #c026d3

700 #a21caf

800 #86198f

900 #701a75

Stop Hex

950 #4a044e

Status: hue family in spec; specific stop confirmed in §5 h06.

4.12 Orange — ZugaForge

Role: ZugaForge studio accent. Hue confirm in §5 (checkpoint h07). Orange conveys build heat and creation energy — appropriate for a forge/fabrication studio.

Stop Hex

50 #fff7ed

100 #ffedd5

200 #fed7aa

300 #fdb74

400 #fb923c

500 #f97316

600 #ea580c

700 #c2410c

800 #9a3412

900 #7c2d12

950 #431407

Status: hue family in spec; specific stop confirmed in §5 h07.

4.13 Indigo — ZugaLearn

Role: ZugaLearn studio accent. Hue confirm in §5 (checkpoint h08). Indigo reads as education and depth — the academic end of the blue spectrum, distinct from ZugaCode (blue) and ZugaCloud (sky).

Stop Hex

50 #eef2ff

100 #e0e7ff

200 #c7d2fe

300 #a5b4fc

400 #818cf8

500 #6366f1

600 #4f46e5

700 #4338ca

800 #3730a3

900 #312e81

950 #1e1b4b

Status: hue family in spec; specific stop confirmed in §5 h08.

4.14 Dark-mode swap mechanics

Tier 1 primitives are immutable — no stop value changes in dark mode. What changes is which stop the Tier 2 semantic token points to. The swap table below covers the Tier 2 roles defined in `tokens-dark.css`.

Light mode		Dark mode
--surface-canvas	slate-50	slate-950
--surface-primary	#ffffff	slate-900
--surface-secondary	slate-100	slate-800
--surface-tertiary	slate-200	slate-700
--surface-inverse	slate-900	slate-50
--surface-brand	lime-50	lime-950
--text-primary	slate-900	slate-50
--text-secondary	slate-700	slate-300
--text-tertiary	slate-500	slate-400
--text-muted	slate-400	slate-500
--text-link	lime-700	lime-400
--text-link-hover	lime-800	lime-300
--border-subtle	slate-200	slate-800
--border-default	slate-300	slate-700
--border-strong	slate-400	slate-600
--feedback-success-subtle	green-100	green-950
--feedback-warn-subtle	amber-100	amber-950
--feedback-danger-subtle	red-100	red-950
--feedback-info-subtle	cyan-100	cyan-950

Accent tokens (`--accent-brand`, `--accent-brand-subtle`, `--accent-brand-strong`, `--accent-fg`) are unchanged between light and dark mode — the same stops hold sufficient contrast on both `--surface-primary` values. See §5 contrast tables for measured ratios.

Dark-mode feedback subtle note: On dark surfaces, the subtle-bg + main-fg pairings (e.g., green-700 on green-950) produce approximately 3.2:1 — below WCAG AA for normal text. The correct pattern on dark surfaces is: use the deep-stop subtle as background, pair with `--text-primary` (slate-50) as foreground, not the `--feedback-* fg` stop. This is formalized in §5 contrast tables.

5. Color in Practice

5. Color in Practice

Authored: 2026-04-25 Mike checkpoints: h01–h09 (hue confirm), acc01–acc21 (studio accent confirm)

Tier 2 semantic tokens map raw ramp stops to meaning. Sub-brand profiles override --accent-* and --density-* only — everything else inherits from the master token layer. All hue-confirm and accent-confirm decisions are tagged for Mike review below.

5.1 Master accent hue

The master Zuga accent hue is [MIKE-CHECKPOINT-h01].

● **MIKE-CHECKPOINT-h01** — confirm master accent hue

Field	Value
Antonio’s draft	#a3e635 (lime-400) as the brand-identity reference stop AND the semantic --accent-brand functional stop. Lime is a light-forward accent — unlike the old cyan-700 default, there is no separate darker semantic shade; lime-400 is used as a filled background with dark (#0a0a0a) foreground on our near-black corp/product surfaces, where it clears AAA. The hue family — lime — is the master brand accent for all surfaces that do not earn a sub-brand override.
Alternates considered	#65a30d (lime-600) — rejected: too somber; loses energy at small sizes and in pill-shaped elements. #bef264 (lime-300) — rejected: fails AA on white, reads as juvenile against slate neutrals. #3b82f6 (blue-500) — rejected: maps to ZugaCode convention; using it for master brand creates collision with studio identity.
Renders on	Corp landing CTAs, ZugaApp primary buttons, ZugaApp navbar active states, focus rings system-wide, ZugaThemes accent pills, master wordmark on dark surfaces
✔ Approve → flip to [MIKE-APPROVED-h01: YYYY-MM-DD] hex=#a3e635 ✎ Modify → edit “Antonio’s draft” field	

5.2 Wellness family — mint hue confirm

The wellness family accent hue is [MIKE-CHECKPOINT-h02].

● **MIKE-CHECKPOINT-h02** — confirm wellness family accent hue (mint)

Field	Value
Antonio’s draft	#14b8a6 (mint-500) as the reference stop for ZugaLife and ZugaHealth. Semantic --accent-brand in wellness profiles uses mint-700 (#0f766e) for AA on white. Mint reads as health, calm, and organic renewal — distinct from lime (brand) and green (success signal). ZugaHealth shares this hue as a sibling to ZugaLife under the wellness family convention.

Field	Value
Alternates considered	#22c55e (green-500) — rejected: green is reserved for --feedback-success; using it for wellness accent creates signal collision. #a3e635 (lime, master) — rejected: wellness studios earn a category override precisely because health is a distinct product category; defaulting to master lime loses studio identity.
Renders on	ZugaLife habit cards, streak indicators, goal progress bars, ZugaHealth metric displays, wellness onboarding UI
☒ Approve → flip to [MIKE-APPROVED-h02: YYYY-MM-DD] hex=#14b8a6  Modify → edit "Antonio's draft" field	

5.3 Creative family — violet hue confirm

The creative family accent hue is [MIKE-CHECKPOINT-h03].

 **MIKE-CHECKPOINT-h03** — confirm creative family accent hue (violet)

Field	Value
Antonio's draft	#8b5cf6 (violet-500) as the reference stop for Zugalmage, ZugaVideo, and ZugaMotion. Semantic --accent-brand in creative profiles uses violet-700 (#6d28d9) for AA on white. Violet reads as visual production, imagination, and creative tooling without the "corporate tech" read of blue. All three creative studios share this hue because their function is unified: media creation and manipulation.
Alternates considered	#a855f7 (Tailwind purple-500) — not in the token inventory; violet covers the same perceptual range without introducing a 14th ramp. #ec4899 (pink) — rejected: too fashion-forward, clashes with ZugaAudio's magenta when the two studios appear together on the same surface.
Renders on	Zugalmage gallery headers, video generation controls in ZugaVideo, motion timeline in ZugaMotion, creative studio nav indicators
☒ Approve → flip to [MIKE-APPROVED-h03: YYYY-MM-DD] hex=#8b5cf6  Modify → edit "Antonio's draft" field	

5.4 ZugaCode — blue hue confirm

The ZugaCode accent hue is [MIKE-CHECKPOINT-h04].

 **MIKE-CHECKPOINT-h04** — confirm ZugaCode accent hue (blue)

Field	Value
Antonio's draft	#3b82f6 (blue-500) as the reference stop for ZugaCode. Semantic --accent-brand in the ZugaCode profile uses blue-700 (#1d4ed8) for AA on white. Blue is the canonical developer-tools convention — VS Code, GitHub, GitLab all operate in this range. Leaning into convention lowers cognitive load for the developer-audience ZugaCode targets.

Field	Value
Alternates considered	#0ea5e9 (sky-500) — rejected: sky is reserved for ZugaCloud; using it for ZugaCode merges two conceptually distinct studios in the user’s eye. #6366f1 (indigo-500) — rejected: indigo is ZugaLearn; education and code-tooling are separate categories.
Renders on	ZugaCode activity feed, diff viewer, commit panel, hook status indicators, code review queue headers
☒ Approve → flip to [MIKE-APPROVED-h04: YYYY-MM-DD] hex=#3b82f6  Modify → edit “Antonio’s draft” field	

5.5 ZugaCloud — sky hue confirm

The ZugaCloud accent hue is [MIKE-CHECKPOINT-h05].

 **MIKE-CHECKPOINT-h05** — confirm ZugaCloud accent hue (sky)

Field	Value
Antonio’s draft	#0ea5e9 (sky-500) as the reference stop for ZugaCloud. Semantic --accent-brand in the ZugaCloud profile uses sky-700 (#0369a1) for AA on white. Sky is lighter and airier than blue — it reads “cloud, infrastructure, distributed” rather than “IDE, terminal, code.” The perceptual distance from blue-500 is enough to distinguish siblings in the nav at a glance.
Alternates considered	#a3e635 (lime, master) — rejected: ZugaCloud would be visually indistinguishable from the master brand, losing studio identity in mixed navigation. #3b82f6 (blue, ZugaCode) — rejected: both studios would appear identical; cloud and code are different product categories.
Renders on	ZugaCloud storage indicators, sync-status chips, infrastructure health panels, ZugaCloud nav active state
☒ Approve → flip to [MIKE-APPROVED-h05: YYYY-MM-DD] hex=#0ea5e9  Modify → edit “Antonio’s draft” field	

5.6 ZugaAudio — magenta hue confirm

The ZugaAudio accent hue is [MIKE-CHECKPOINT-h06].

 **MIKE-CHECKPOINT-h06** — confirm ZugaAudio accent hue (magenta)

Field	Value
Antonio’s draft	#d946ef (magenta-500, sourced from Tailwind fuchsia) as the reference stop for ZugaAudio. Semantic --accent-brand in the ZugaAudio profile uses magenta-700 (#a21caf) for AA on white. Magenta is the standard music and audio production convention — Ableton, Spotify Wrapped, SoundCloud all operate in the pink/magenta spectrum. It pops against dark studio surfaces where audio waveforms and mixing UI live.

Field	Value
Alternates considered	#ec4899 (Tailwind pink-500) — not in the token inventory; magenta covers the same perceptual territory. #8b5cf6 (violet) — rejected: violet is the creative family (video/image/motion); audio is a separate discipline that deserves its own signal color.
Renders on	ZugaAudio waveform highlights, track level indicators, audio studio nav active state, recording session status badges
☒ Approve → flip to [MIKE-APPROVED-h06: YYYY-MM-DD] hex=#d946ef  Modify → edit "Antonio's draft" field	

5.7 ZugaForge — orange hue confirm

The ZugaForge accent hue is [MIKE-CHECKPOINT-h07].

 **MIKE-CHECKPOINT-h07** — confirm ZugaForge accent hue (orange)

Field	Value
Antonio's draft	#f97316 (orange-500) as the reference stop for ZugaForge. Semantic --accent-brand in the ZugaForge profile uses orange-700 (#c2410c) for AA on white. Orange conveys build heat, fabrication energy, and creation momentum — appropriate for a forge/creation registry studio. It also aligns with the "build" tool convention (Buildkite, Netlify deploy indicators).
Alternates considered	#f59e0b (amber-500) — rejected: amber is --feedback-warn; sharing the hue with warning states would make ZugaForge UI feel like something is wrong. #ef4444 (red-500) — rejected: red is --feedback-danger and ZugaShield; two studios on red creates collision.
Renders on	ZugaForge creation registry items, build-pipeline progress indicators, forge studio nav active state, creation-run status badges
☒ Approve → flip to [MIKE-APPROVED-h07: YYYY-MM-DD] hex=#f97316  Modify → edit "Antonio's draft" field	

5.8 ZugaLearn — indigo hue confirm

The ZugaLearn accent hue is [MIKE-CHECKPOINT-h08].

 **MIKE-CHECKPOINT-h08** — confirm ZugaLearn accent hue (indigo)

Field	Value
Antonio's draft	#6366f1 (indigo-500) as the reference stop for ZugaLearn. Semantic --accent-brand in the ZugaLearn profile uses indigo-700 (#4338ca) for AA on white. Indigo reads as education, depth, and academic gravity — the scholarly end of the blue spectrum. It sits between blue (ZugaCode) and violet (creative family) perceptually, creating a coherent nav spectrum without collision.
Alternates considered	#3b82f6 (blue, ZugaCode) — rejected: learning and coding are separate activities in the product; conflating their studio colors undermines the distinction. #8b5cf6 (violet,

Field	Value
	creative family) — rejected: violet is reserved for media-creation tools; education is a different product category.
Renders on	ZugaLearn course progress bars, lesson completion badges, curriculum nav active state, quiz result displays
☒	Approve → flip to [MIKE-APPROVED-h08: YYYY-MM-DD] hex=#6366f1  Modify → edit "Antonio's draft" field

5.9 Markets family — emerald hue confirm

The markets family accent hue is [MIKE-CHECKPOINT-h09].

 **MIKE-CHECKPOINT-h09** — confirm markets family accent hue (emerald)

Field	Value
	#10b981 (emerald-500) as the reference stop for ZugaTrader and ZugaTraderOverlay. Semantic --accent-brand in the markets profile uses emerald-700 (#047857) for AA
Antonio's draft	on white. Emerald reads as "money and markets" — distinct from green-500 (which is -feedback-success). Bloomberg terminals, trading platforms, and financial dashboards universally operate in this green-to-emerald range. ZugaTraderOverlay inherits this as a sibling because it surfaces the same data in-game.
Alternates considered	#22c55e (green-500) — rejected: green-500 is --feedback-success; markets accent on the same stop causes the entire ZugaTrader UI to read as a permanent success state. #a3e635 (lime, master) — rejected: markets is a high-conviction category that earns its own override; defaulting to master lime loses the financial-platform signal.
Renders on	ZugaTrader P&L indicators, position cards, watchlist rows, order execution buttons, ZugaTraderOverlay ticker displays
☒	Approve → flip to [MIKE-APPROVED-h09: YYYY-MM-DD] hex=#10b981  Modify → edit "Antonio's draft" field

5.10 Sub-brand accent assignments

The table below maps every consumer, product surface, and internal tool to its accent ramp. Family logic: wellness siblings share mint; creative siblings share violet; markets siblings share emerald. Surfaces that do not earn a category convention default to master lime. All rows are tagged for Mike confirmation.

acc01 — zugatechnologies-site (corp)

The zugatechnologies.com corporate site accent assignment is [MIKE-CHECKPOINT-acc01].

 **MIKE-CHECKPOINT-acc01** — confirm corporate site accent assignment

Field	Value
Antonio's draft	lime.* (master). The corporate landing page is brand identity, not a studio. It does not represent a product category, so it does not earn a category override. Master lime

Field	Value
	is correct — it IS the brand.
Alternates considered	Own distinct hue — rejected: the corporate site is the brand home; it should be the fullest expression of master lime, not a sub-brand variant. #65a30d (lime-600) as a “more serious corporate” choice — rejected: lime-400 already serves as both the identity reference and the functional --accent-brand stop; the system doesn’t need a separate darker corporate shade.
Renders on	zugatechnologies.com CTAs, nav active states, hover states, link color
field	 Approve → flip to [MIKE-APPROVED-acc01: YYYY-MM-DD]  Modify → edit “Antonio’s draft”
	acc02 — ZugaApp/frontend shell (product-shell)
	The ZugaApp product shell accent assignment is [MIKE-CHECKPOINT-acc02].
	 MIKE-CHECKPOINT-acc02 — confirm ZugaApp shell accent assignment

Field	Value
Antonio’s draft	lime.* (master). The ZugaApp shell is the food court — the host surface that aggregates all studios. Its accent should read as “home base,” which is master lime. Studio-specific accents appear inside each studio’s context; the shell frame stays on brand.
Alternates considered	Own distinct hue to differentiate “product” from “corp” — rejected: the shell and the corp site are different surfaces but the same brand; splitting them creates inconsistency at the identity level. Studio-rotating accent (changes based on which studio is active) — rejected: introduces unpredictable chrome shifts that confuse spatial navigation.
Renders on	ZugaApp global nav active states, sidebar indicators, primary action buttons in the shell layer, ZugaApp token wallet display
field	 Approve → flip to [MIKE-APPROVED-acc02: YYYY-MM-DD]  Modify → edit “Antonio’s draft”
	acc03 — ZugaLife (studio-life)
	The ZugaLife accent assignment is [MIKE-CHECKPOINT-acc03].
	 MIKE-CHECKPOINT-acc03 — confirm ZugaLife accent assignment

Field	Value
Antonio’s draft	mint.* (mint-500 reference, mint-700 semantic stop). ZugaLife is the wellness studio — habit tracking, meditation, addiction recovery. Mint reads as health, calm, and organic renewal. The wellness category earns an override because it is perceptually and emotionally distinct from the tech-forward master lime.

Field	Value
Alternates considered	Master lime — rejected: wellness surfaces should feel softer and more biologically grounded than the bright acid punch of brand lime; the category earns the distinction. Green-500 — rejected: green is --feedback-success, not a studio accent; signal collision would make all ZugaLife UI feel like a constant success state.
Renders on	ZugaLife habit cards, streak fire indicators, goal rings, meditation session UI, addiction tracker progress
 Approve → flip to [MIKE-APPROVED-acc03: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc04 — ZugaHealth (studio-health)	
The ZugaHealth accent assignment is [MIKE-CHECKPOINT-acc04].	

 MIKE-CHECKPOINT-acc04 — confirm ZugaHealth accent assignment

Field	Value
Antonio’s draft	mint.* (sibling to ZugaLife, same stop references). ZugaHealth is the clinical/biometric sibling — Fitbit sync, health metrics, body data. It shares mint because both studios are in the wellness family; a user moving between them should not experience a color jump. Same hue, same semantic stops as acc03.
Alternates considered	Distinct wellness-adjacent hue — rejected: we defined the wellness family to mean exactly this: siblings share a hue. Splitting ZugaHealth to a different color breaks the family grouping that makes the product map navigable. Master lime — rejected: same rationale as acc03 above.
Renders on	ZugaHealth metric charts, Fitbit sync status, heart rate displays, health dashboard nav active state
 Approve → flip to [MIKE-APPROVED-acc04: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc05 — ZugaTrader (studio-trader)	
The ZugaTrader accent assignment is [MIKE-CHECKPOINT-acc05].	

 MIKE-CHECKPOINT-acc05 — confirm ZugaTrader accent assignment

Field	Value
Antonio’s draft	emerald.* (emerald-500 reference, emerald-700 semantic stop). ZugaTrader is the markets/financial studio. Emerald is the financial-platform convention; it signals money and markets without confusing with --feedback-success (green). The markets category earns an override because financial data visualization lives in a different emotional register than the brand.
Alternates considered	Master lime — rejected: the markets override is high-conviction; lime reads as the Zuga house color, not “financial terminal.” Green-500 — rejected: same feedback-

Field	Value
	collision problem as described in h09 above.
Renders on	ZugaTrader P&L rows, position cards, order execution primary button, watchlist active indicators, TA signal badges
☒ Approve → flip to [MIKE-APPROVED-acc05: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc06 — Zugalmage (studio-image)	
The Zugalmage accent assignment is [MIKE-CHECKPOINT-acc06].	
 MIKE-CHECKPOINT-acc06 — confirm Zugalmage accent assignment	

Field	Value
Antonio’s draft	violet.* (violet-500 reference, violet-700 semantic stop). Zugalmage is an AI image-generation and editing studio. Violet reads as creativity and visual imagination. It leads the creative family — ZugaVideo and ZugaMotion are siblings under violet.
Alternates considered	Master lime — rejected: creative studios earn an override; lime is brand identity, not creative-tools identity. Magenta — rejected: magenta is ZugaAudio’s signal; image and audio are distinct creative disciplines that need distinct signals.
Renders on	Zugalmage gallery headers, generation prompt controls, image editing toolbar, composable slot indicators
☒ Approve → flip to [MIKE-APPROVED-acc06: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc07 — ZugaVideo (studio-video)	
The ZugaVideo accent assignment is [MIKE-CHECKPOINT-acc07].	
 MIKE-CHECKPOINT-acc07 — confirm ZugaVideo accent assignment	

Field	Value
Antonio’s draft	violet.* (sibling to Zugalmage, same stop references). ZugaVideo is the video generation and wallpaper studio. It shares violet because it is in the creative family — a user who sees violet in Zugalmage will immediately recognize the creative family context in ZugaVideo.
Alternates considered	Distinct creative-adjacent hue per studio — rejected: the creative family is defined to have a shared hue; per-studio splits would make the color system feel arbitrary rather than principled. Master lime — rejected: creative studios earn the override.
Renders on	ZugaVideo provider cards, video generation controls, wallpaper template browser, video pipeline status
☒ Approve → flip to [MIKE-APPROVED-acc07: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	

acc08 — ZugaMotion (studio-motion)

The ZugaMotion accent assignment is [MIKE-CHECKPOINT-acc08].

● **MIKE-CHECKPOINT-acc08** — confirm ZugaMotion accent assignment

Field	Value
Antonio's draft	violet.* (sibling to Zugalmage and ZugaVideo, same stop references). ZugaMotion handles animation and motion design. It completes the creative family triad under violet. Same rationale as acc07 — the family grouping creates navigable product cohesion.
Alternates considered	Distinct hue for motion — rejected: we have three creative siblings (image, video, motion) and three is exactly the right number to justify a named family with a shared hue. Fragmenting to individual hues removes the family signal.
Renders on	ZugaMotion animation timeline, keyframe editor, preview panel controls, motion studio nav active state

✓ Approve → flip to [MIKE-APPROVED-acc08: YYYY-MM-DD] ✎ Modify → edit "Antonio's draft" field

acc09 — ZugaCode (studio-code)

The ZugaCode accent assignment is [MIKE-CHECKPOINT-acc09].

● **MIKE-CHECKPOINT-acc09** — confirm ZugaCode accent assignment

Field	Value
Antonio's draft	blue.* (blue-500 reference, blue-700 semantic stop). ZugaCode is the developer-tools studio — activity tracking, commit hooks, code review queue. Blue is the canonical dev-tools convention; leaning into it reduces cognitive load for the developer audience and signals "this is where code happens."
Alternates considered	Master lime — rejected: the dev-tools convention is blue; using lime here would mean ZugaCode looks like a generic Zuga surface rather than a purpose-built code studio. Indigo — rejected: indigo is ZugaLearn; code and education are distinct.
Renders on	ZugaCode activity feed, diff viewer chrome, commit panel status, hook failure notifications, review queue badges

✓ Approve → flip to [MIKE-APPROVED-acc09: YYYY-MM-DD] ✎ Modify → edit "Antonio's draft" field

acc10 — ZugaCloud (studio-cloud)

The ZugaCloud accent assignment is [MIKE-CHECKPOINT-acc10].

● **MIKE-CHECKPOINT-acc10** — confirm ZugaCloud accent assignment

Field	Value
Antonio's draft	sky.* (sky-500 reference, sky-700 semantic stop). ZugaCloud is the file storage and sync studio. Sky reads as "infrastructure and distributed systems" — lighter and more

Field	Value
	expansive than the IDE-register of blue. The sky/blue sibling pair mirrors the cloud/code conceptual pair in the product.
Alternates considered	Blue — rejected: ZugaCloud and ZugaCode would be visually indistinguishable; cloud infrastructure and code tooling are different product categories. Master lime — rejected: sky earns the override because cloud is a distinct category with an established color convention.
Renders on	ZugaCloud storage bars, sync-status chips, infrastructure health panels, cloud nav active state
 Approve → flip to [MIKE-APPROVED-acc10: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc11 — ZugaAudio (studio-audio)	
The ZugaAudio accent assignment is [MIKE-CHECKPOINT-acc11].	
 MIKE-CHECKPOINT-acc11 — confirm ZugaAudio accent assignment	

Field	Value
Antonio’s draft	magenta.* (magenta-500 reference, magenta-700 semantic stop). ZugaAudio is the music and audio production studio. Magenta is the music convention — waveform editors and production tools from Ableton to SoundCloud operate in the pink/magenta range. It pops against dark studio surfaces where audio visualization lives.
Alternates considered	Violet — rejected: violet is the creative family (visual media); audio is a distinct discipline with its own established color convention. Master lime — rejected: audio earns the override; the music convention is magenta, not lime.
Renders on	ZugaAudio waveform highlights, track level indicators, studio recording session status, audio nav active state
 Approve → flip to [MIKE-APPROVED-acc11: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc12 — ZugaForge (studio-forge)	
The ZugaForge accent assignment is [MIKE-CHECKPOINT-acc12].	
 MIKE-CHECKPOINT-acc12 — confirm ZugaForge accent assignment	

Field	Value
Antonio’s draft	orange.* (orange-500 reference, orange-700 semantic stop). ZugaForge is the creation registry and build-pipeline studio. Orange conveys build heat and fabrication energy — the color of molten metal and combustion, which is exactly the metaphor. Build tools (Buildkite, deploy pipelines) use orange and amber as their primary signal color.

Field	Value
Alternates considered	Amber — rejected: amber is --feedback-warn; the entire ZugaForge UI would read as a permanent warning state. Red — rejected: red is --feedback-danger and ZugaShield; two studios on red collapses the signal.
Renders on	ZugaForge creation registry item cards, build-run progress indicators, forge pipeline status badges, forge nav active state
 Approve → flip to [MIKE-APPROVED-acc12: YYYY-MM-DD]  Modify → edit "Antonio's draft" field	
acc13 — ZugaThemes (studio-themes)	
The ZugaThemes accent assignment is [MIKE-CHECKPOINT-acc13].	
 MIKE-CHECKPOINT-acc13 — confirm ZugaThemes accent assignment	

Field	Value
Antonio's draft	lime.* (master). ZugaThemes is the design-token marketplace — it sells visual customization to other studios. Its own UI should be on master lime to signal "this is a ZugaApp-layer product, not a studio with its own personality." Theming other studios is its function; it should not impose its own color identity.
Alternates considered	Own distinct hue (e.g., coral or gold) — rejected: ZugaThemes does not have a product category convention; it is a marketplace layer that does not earn an override. Rotating to reflect the active theme for sale — rejected: the chrome of the marketplace should be stable; previewed themes render in contained preview slots, not in the chrome.
Renders on	ZugaThemes marketplace shell, theme card accent pills, purchase CTA buttons, marketplace nav active state
 Approve → flip to [MIKE-APPROVED-acc13: YYYY-MM-DD]  Modify → edit "Antonio's draft" field	
acc14 — ZugaLearn (studio-learn)	
The ZugaLearn accent assignment is [MIKE-CHECKPOINT-acc14].	
 MIKE-CHECKPOINT-acc14 — confirm ZugaLearn accent assignment	

Field	Value
Antonio's draft	indigo.* (indigo-500 reference, indigo-700 semantic stop). ZugaLearn is the education and curriculum studio. Indigo reads as academic depth and intellectual gravity — the scholarly end of the blue spectrum. It is perceptually distinct from ZugaCode (blue) and ZugaCloud (sky), creating a coherent blue-family spectrum across three related but different studios.
Alternates considered	Blue — rejected: blue is ZugaCode; learning and coding are distinct product experiences. Violet — rejected: violet is the creative family; education is not creative-

Field	Value
	media production.
Renders on	ZugaLearn course progress bars, lesson completion badges, quiz result displays, curriculum nav active state
☒ Approve → flip to [MIKE-APPROVED-acc14: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc15 — ZugaCraft (studio-craft)	
The ZugaCraft accent assignment is [MIKE-CHECKPOINT-acc15].	
 MIKE-CHECKPOINT-acc15 — confirm ZugaCraft accent assignment	

Field	Value
Antonio’s draft	lime.* (master) — pending Antonio personality input before Phase 0 seal. ZugaCraft’s product category is not yet defined precisely enough to earn a category-convention override. Master lime is the correct default under spec rule: “defaults to master per spec rule.” If Antonio identifies a clear product category for ZugaCraft (e.g., handmade goods marketplace, crafting game overlay) before Phase 0 closes, this checkpoint should be revisited and may earn its own hue.
Alternates considered	Own distinct hue — rejected at this time: ZugaCraft does not yet have a defined product category strong enough to justify pulling a new convention from thin air; conventions are earned by product identity, not invented. Violet (creative family) — considered: if ZugaCraft ends up being a digital-creation tool, violet is a candidate; Antonio to confirm.
Renders on	ZugaCraft studio UI — TBD pending product definition; assumes standard master-lime chrome until category is specified
☒ Approve → flip to [MIKE-APPROVED-acc15: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc16 — ZugaNews (studio-news)	
The ZugaNews accent assignment is [MIKE-CHECKPOINT-acc16].	

 **MIKE-CHECKPOINT-acc16** — confirm ZugaNews accent assignment

Field	Value
Antonio’s draft	lime.* (master). ZugaNews is the news and intelligence studio. Journalism and news platforms historically use neutral color systems — black, white, and a single accent; the accent is rarely a “hot” color. Master lime is the appropriate default because it reads as informational and credible without the emotional charge of amber (warn) or red (danger). ZugaNews does not have a proprietary product-category convention.
Alternates considered	Own distinct hue — rejected: news/journalism does not have a universal studio-accent convention strong enough to override; defaulting to master keeps the brand coherent

Field	Value
	without an arbitrary color decision. Slate-900 (near-black, very journalistic) — rejected: the design system does not support monochrome studio identities; the neutral-slate story is handled by the surface system.
Renders on	ZugaNews article headers, source badges, intelligence summary cards, news nav active state
☒ Approve → flip to [MIKE-APPROVED-acc16: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc17 — ZugaGamerOverlay (overlay-gamer)	
The ZugaGamerOverlay accent assignment is [MIKE-CHECKPOINT-acc17].	

 **MIKE-CHECKPOINT-acc17** — confirm ZugaGamerOverlay accent assignment

Field	Value
Antonio’s draft	lime.* (master). The gaming overlay is a neutral surface — it surfaces data and voice commands, but does not advocate for a game genre or platform color convention. Lime on dark overlay backgrounds gives a clean “HUD” read without the aggression of red/orange overlays that feel like threat indicators. Gaming HUDs commonly use bright greens as neutral data-display colors.
Alternates considered	Aggressive gaming palette (neon green or hot red) — rejected: neon green reads as “RGB gaming peripheral” not “intelligent assistant overlay”; hot red conflicts with danger/threat signals in games. Master lime is the neutral choice here, and neutral is right for an information-display layer.
Renders on	ZugaGamerOverlay HUD panel borders, wake-word indicator, TTS display, stat readouts, dismiss controls
☒ Approve → flip to [MIKE-APPROVED-acc17: YYYY-MM-DD]  Modify → edit “Antonio’s draft” field	
acc18 — ZugaTraderOverlay (overlay-trader)	
The ZugaTraderOverlay accent assignment is [MIKE-CHECKPOINT-acc18].	

 **MIKE-CHECKPOINT-acc18** — confirm ZugaTraderOverlay accent assignment

Field	Value
Antonio’s draft	emerald.* (sibling to ZugaTrader, same stop references). ZugaTraderOverlay is a game-overlay surface for ZugaTrader data — it shows the same financial data in a HUD format while gaming. It should be immediately recognizable as “ZugaTrader data” from the accent color alone. Matching ZugaTrader’s emerald achieves this without extra visual vocabulary.
Alternates considered	Master lime (neutral overlay convention from acc17) — rejected: ZugaTraderOverlay is not a neutral tool; it is specifically a financial data display, and the emerald markets

Field	Value
	accent is load-bearing for that association. Own distinct hue — rejected: the overlay and its parent studio should share an accent; introducing a third color splits the product identity.
Renders on	ZugaTraderOverlay position cards, P&L ticker, order status chips, financial HUD panel chrome
☒ Approve → flip to [MIKE-APPROVED-acc18: YYYY-MM-DD]  Modify → edit "Antonio's draft" field	
acc19 — ZugaShield (studio-shield)	
The ZugaShield accent assignment is [MIKE-CHECKPOINT-acc19].	
 MIKE-CHECKPOINT-acc19 — confirm ZugaShield accent assignment	

Field	Value
Antonio's draft	red.* (red-500 reference, red-700 semantic stop for interactive elements). ZugaShield is the security studio — threat detection, account monitoring, shield events. Red is the universal security-signal color. It carries the same weight as --feedback-danger intentionally: security alerts ARE danger states. The studio UI should feel like a monitoring dashboard that takes its work seriously.
Alternates considered	A "softer" security color (slate, indigo) — rejected: security needs to read as "this matters" at a glance; a neutral or educational hue removes that urgency. Master lime — rejected: ZugaShield earning the danger-signal red is the correct override; lime here would make security surface feel like a generic Zuga tool.
Renders on	ZugaShield threat alerts, shield event severity badges, account health indicators, security nav active state
☒ Approve → flip to [MIKE-APPROVED-acc19: YYYY-MM-DD]  Modify → edit "Antonio's draft" field	
acc20 — ZugaClaw (internal-claw)	
The ZugaClaw accent assignment is [MIKE-CHECKPOINT-acc20].	
 MIKE-CHECKPOINT-acc20 — confirm ZugaClaw accent assignment	

Field	Value
Antonio's draft	lime.* (master). ZugaClaw is an internal infrastructure tool — the third-party AI provider bridge layer. Internal tooling defaults to master lime for cohesion; it does not have a user-facing product identity that would earn a category override. Engineers using ZugaClaw are Zuga engineers; the product context is "Zuga infrastructure" not a distinct studio.
Alternates considered	Own distinct hue to signal "internal / restricted" — rejected: creating a special color for internal tools adds a new semantic layer that does not exist in the token system;

Field	Value
	internal vs external is handled by access control, not color. Master lime affirms that internal tools belong to the same design system.
Renders on	ZugaClaw internal admin UI elements (if any surface-level UI exists), internal tooling chrome
✓ Approve → flip to [MIKE-APPROVED-acc20: YYYY-MM-DD] ✎ Modify → edit “Antonio’s draft”	
field	
acc21 — BugaBot (internal-bugabot)	
The BugaBot internal tooling accent assignment is [MIKE-CHECKPOINT-acc21].	
● MIKE-CHECKPOINT-acc21 — confirm BugaBot accent assignment	

Field	Value
Antonio’s draft	lime.* (master). BugaBot is internal tooling — Antonio’s personal automation and development assistant layer. Like ZugaClaw (acc20), internal tooling defaults to master lime. BugaBot has no user-facing product surface; the accent appears only in internal dashboards and developer tooling. Cohesion with the master brand is the right call for tools with no external audience.
Alternates considered	Own distinct hue to differentiate “developer persona” from product brand — rejected: the developer persona already has a dedicated hue (blue, ZugaCode); using blue for BugaBot would conflate internal automation with the ZugaCode studio product. Master lime keeps internal tools clearly in the Zuga system without studio collision.
Renders on	BugaBot internal admin surfaces, developer tooling UI if rendered, internal status dashboards
✓ Approve → flip to [MIKE-APPROVED-acc21: YYYY-MM-DD] ✎ Modify → edit “Antonio’s draft”	
field	

5.11 Contrast tables

All ratios below are measured using the WCAG 2.1 relative-luminance formula. Ratios are stated as measured values, not thresholds. AA normal text requires 4.5:1; AA large text / UI components require 3:1; AAA normal text requires 7:1.

Light-mode — key pairings

Foreground	Background	Measured ratio	Clears
--text-primary slate-900 #0f172a	--surface-primary white #ffffff	≈17.85:1	AAA
--text-primary slate-900 #0f172a	--surface-canvas slate-50 #f8fafc	≈16.75:1	AAA

Foreground	Background	Measured ratio		Clears
--text-secondary slate-700 #334155	--surface-primary white #ffffff	≈10.65:1	AAA	
--text-link lime-700 #4d7c0f	--surface-primary white #ffffff	≈5.00:1	AA	
--accent-brand lime-400 #a3e635 (filled bg)	--accent-fg dark #0a0a0a (button fg)	≈13.1:1	AAA	
--accent-fg dark #0a0a0a	--accent-brand lime-400 #a3e635 (button bg)	≈13.1:1	AAA	
--feedback-success green-700 #15803d	white #ffffff	≈5.02:1	AA	
--feedback-warn amber-600 #d97706	--surface-primary white #ffffff	≈3.12:1	AA large/UI only — warn text uses --text-primary (slate-900) as fg, not white	
--feedback-danger red-600 #dc2626	white #ffffff	≈4.83:1	AA	

Known failure: cyan-500 (#06b6d4) against white (#ffffff) ≈ 2.43:1 — fails AA for all text sizes. This remains true of the raw cyan ramp, but cyan-500 is no longer a brand reference stop — cyan is retired as the master accent and now backs only --feedback-info (see §2.4). Never place cyan-500 text on a white surface.

Dark-mode — key pairings

Foreground	Background	Measured ratio	Clears
--text-primary slate-50 #f8fafc	--surface-primary slate-900 #0f172a	≈17.06:1	AAA
--text-secondary slate-300 #cbd5e1	--surface-primary slate-900 #0f172a	≈12.02:1	AAA
--text-link lime-400 #a3e635	--surface-primary slate-900 #0f172a	≈11.84:1	AAA
--accent-brand lime-400 #a3e635 (filled bg)	--accent-fg dark #0a0a0a (button fg)	≈13.1:1	AAA
--accent-fg dark #0a0a0a	--accent-brand lime-400 #a3e635	≈13.1:1	AAA

Dark-mode feedback subtle note (from tokens-dark.css): Subtle-bg + main-fg pairings like green-700 on green-950 produce approximately 3.2:1 — below AA for normal text. On dark

surfaces, subtle alert boxes must pair the deep-stop subtle background with `--text-primary` (slate-50) as foreground, not the `--feedback-*fg` stop.

5.12 Do / Don't

Do:

Use `--accent-brand` (lime-400 #a3e635 in master profile, always with dark foreground) for all interactive primary elements.

Override `--accent-brand` at the profile level for studios that have earned a category convention.

Use `--feedback-*` tokens exclusively for success/warn/danger/info states — not for decorative accent.

Pair `--accent-fg` (white) as the text color on any `--accent-brand` background.

Don't:

Place cyan-500 (#06b6d4) text on white or light surfaces — it fails AA.

Use emerald for anything other than markets-family elements — it will be read as financial data.

Use red outside of danger/ZugaShield contexts — it carries the danger semantic system-wide.

Use amber for studio accents — amber is the exclusive property of `--feedback-warn`.

6. Typography

6. Typography

Authored: 2026-04-25 Mike checkpoints: typ01 (custom display face)

Typography is functional first. Every scale decision below serves legibility and information density — not visual novelty. One checkpoint (typ01) covers the deferred custom display face; everything else is Antonio-final.

6.1 Font stacks

Three stacks are in play. All values are sourced from packages/design-tokens/src/tokens.css.

Sans (body, UI, labels)

```
system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif
```

Token: --font-family-sans

System-first. No web font request, no FOIT, no external dependency. The stack renders the OS native UI font on every platform: San Francisco on macOS/iOS, Segoe UI on Windows, Roboto on Android. Consistent with ZugaApp’s posture as a desktop-first progressive web app.

Mono (code, tokens, data)

```
ui-monospace, SFMono-Regular, "SF Mono", Menlo, Consolas, "Liberation Mono", monospace
```

Token: --font-family-mono

Used in: ZugaCode diff viewer, API key displays, ZugaTrader order IDs, any literal value that must be read character-by-character. ui-monospace is the modern entry that picks the OS native mono (SF Mono on Apple, Cascadia Code on modern Windows). Fallback chain covers pre-ui-monospace environments.

Display (hero copy, marketing headings)

The custom display typeface is [MIKE-CHECKPOINT-typ01].

● **MIKE-CHECKPOINT-typ01** — confirm display typeface direction

Field	Value
	Deferred to Phase 2. --font-family-display currently aliases --font-family-sans (system stack). Rationale: introducing a web font in Phase 0 adds a network dependency and FOUT risk before the design system is stable. The system-font hero headings already read as intentional on modern OS renderers. Phase 2 will evaluate a geometric sans (candidates: Inter Display, Geist, or a custom variable font at weights 600–800) — but no font is selected until Phase 0 is sealed.
Antonio’s draft	
Alternates considered	Inter (Google Fonts) — considered: excellent metrics, very legible; rejected for Phase 0 because every SaaS product uses Inter and differentiation matters at the display size; decision deferred not rejected. Geist (Vercel) — considered: geometric, modern;

Field	Value
	rejected for Phase 0 because brand association with Vercel is a concern given Zuga's infrastructure choices. Custom variable font — considered: maximum differentiation; rejected for Phase 0 because commissioning takes time and budget not yet allocated.
	Corp landing H1/H2, ZugaApp marketing hero, ZugaThemes promotional banners —
Renders on	currently renders as system-ui at weight 700; will be swapped to a custom face once Phase 2 decision lands
☒ Approve → flip to [MIKE-APPROVED-typ01: YYYY-MM-DD]  Modify → edit "Antonio's draft"	

6.2 Type scale

The scale uses a modular ratio of approximately 1.2 (minor third), anchored at 1rem / 16px base. Values are sourced from `packages/design-tokens/src/tokens.css`.

Token	rem	px	Role
--type-scale-xs	0.75rem	12px	Caption, timestamp, badge label
--type-scale-sm	0.875rem	14px	Secondary body, tooltip, label
--type-scale-base	1rem	16px	Primary body text
--type-scale-lg	1.125rem	18px	Lead paragraph, emphasized body
--type-scale-x1	1.25rem	20px	Card title, section subtitle
--type-scale-2x1	1.5rem	24px	H3 / sub-section heading
--type-scale-3x1	1.875rem	30px	H2 / content heading
--type-scale-4x1	2.25rem	36px	H1 / page heading
--type-scale-5x1	3rem	48px	Display small (hero sub)
--type-scale-6x1	3.75rem	60px	Display medium (marketing H1)
--type-scale-7x1	4.5rem	72px	Display large (hero feature callout)

Rendered examples (approximate visual reference):

xs (12px): Used for timestamps, badges

sm (14px): Secondary body — sidebar annotations

base (16px): Primary body — this line is 16px

lg (18px): Lead paragraph

x1 (20px): Card title

2x1 (24px): **Sub-section heading**

3x1 (30px): **Content heading**

4x1 (36px): **Page heading**

5x1 (48px): **Display small**

6.3 Weight inventory

Token	Value	Role
--font-weight-regular	400	Body text, secondary labels
--font-weight-medium	500	Emphasized body, nav items
--font-weight-semibold	600	Card titles, UI labels, H3
--font-weight-bold	700	H1/H2, CTA button labels, critical alerts

Weight pairing rules:

Body text (--type-scale-base or smaller): regular (400).

Interactive labels and nav: medium (500) — enough weight to signal interactivity without the heaviness of bold.

Headings at --type-scale-2x1 and above: semibold (600) or bold (700). Bold is reserved for H1 and primary CTAs.

Never use a weight below 400 — the system stack has no thin/light variants that render consistently across OS renderers.

6.4 Line-height

Token	Value	Role
--font-line-height-tight	1.1	Display headings (5xl–7xl) — optical tracking at large sizes
--font-line-height-snug	1.25	H1–H3, card titles — compact but not cramped
--font-line-height-normal	1.5	Body text, UI copy — WCAG SC 1.4.12 compliant at 1.5
--font-line-height-relaxed	1.625	Long-form reading surfaces (docs, legal copy, ZugaLearn lesson body)

Rationale: Body at 1.5 satisfies WCAG Success Criterion 1.4.12 (Text Spacing) without override.

Display headings at 1.1 prevent exaggerated gaps between wrapped lines at 48–72px that make multi-line heroes look broken. Relaxed at 1.625 is appropriate only for extended reading (ZugaLearn, ZugaNews article body) — using it in UI copy inflates layout without benefit.

6.5 Typography pairing table

Common surface-to-token pairings for reference. These are not exhaustive — they document the load-bearing cases.

Surface	Size	token	Weight	Line height
Corp landing H1	7x1		bold	tight
Corp landing H2	4x1		bold	snug
ZugaApp page heading (H1)	4x1		bold	snug
ZugaApp section heading (H2)	3x1		semibold	snug
ZugaApp card title	x1		semibold	snug
Body copy (all surfaces)	base		regular	normal
Studio sidebar label	sm		medium	normal

Surface	Size token	Weight	Line height
Badge / chip	xs	semibold	tight
ZugLearn lesson body	base	regular	relaxed
ZugaCode diff line	sm (mono)	regular	normal
Tooltip	sm	regular	snug

6.6 Typography do / don't

Do:

Use `--font-family-mono` for any literal value — API keys, commit SHAs, token IDs, order numbers.

Use `--font-line-height-normal` (1.5) for all body text.

Pair semibold/bold with `--type-scale-2x1` and above.

Don't:

Mix `--font-family-display` (currently the same as sans) with a distinct web-font size before `typ01` is resolved — the swap needs to happen system-wide, not per-component.

Use `--type-scale-xs` for body text — 12px body fails WCAG SC 1.4.4 (Resize Text) at default zoom on some platforms.

Set body text weight above 500 — medium body text reads as stressed copy, not normal reading state.

7. Spacing & Layout

7. Spacing & Layout

Authored: 2026-04-25 Mike checkpoints: none (Antonio-final)

All spacing and layout decisions in this section are fully resolved. No checkpoints. Values are sourced from `packages/design-tokens/src/tokens.css`.

7.1 Spacing scale — rationale

The scale is **4-based** (base unit = 4px = 0.25rem). Every step is a multiple or fraction of 4. This is the same base unit as Tailwind, Material, and most contemporary design systems — which matters because it means spacing values from any of those systems map without conversion.

Why 4px and not 8px? An 8-base system skips half-steps that UI components actually need: a 2px inset between a button and its outline ring, a 6px gap between an icon and a label. A 4-base system with fractional steps (0.5, 1.5) covers those cases without breaking the rhythm.

The scale is intentionally **sparse at the high end** — jumps from 16 → 20 → 24 → 32 are wide enough that they produce visible, purposeful rhythm breaks rather than incremental drift.

7.2 Spacing scale — full table

Token	rem	px	Use
--space-0	0	0px	Reset / flush
--space-0_50	0.125rem	2px	Focus ring inset, micro gap between icon + badge pip
--space-1	0.25rem	4px	Icon internal padding, badge horizontal inset
--space-1_50	0.375rem	6px	Icon + label gap, tight inline elements
--space-2	0.5rem	8px	Input internal padding (tight), compact row gap
--space-3	0.75rem	12px	Chip / tag padding, button padding Y
--space-4	1rem	16px	Default gap — card padding, form field spacing
--space-5	1.25rem	20px	Button padding X, list item padding
--space-6	1.5rem	24px	Card padding (comfortable), modal header padding
--space-8	2rem	32px	Section inner gap, sidebar item block padding
--space-10	2.5rem	40px	Large card padding, content well side padding (mobile)
--space-12	3rem	48px	Content section top/bottom padding (mobile)
--space-16	4rem	64px	Default section spacing on desktop
--space-20	5rem	80px	Section spacing between major content blocks
--space-24	6rem	96px	Hero section padding top/bottom
--space-32	8rem	128px	Maximum section breathing room (corp landing sections)

7.3 Semantic spacing tokens

The token layer adds semantic spacing values on top of the raw scale. These are the values to reference in component code — not the raw `--space-*` tokens.

Token	Maps to	Use
<code>--spacing-section</code>	<code>--space-16</code> (64px)	Default top/bottom padding between page sections
<code>--spacing-section-sm</code>	<code>--space-12</code> (48px)	Reduced section padding on compact views / mobile
<code>--spacing-section-lg</code>	<code>--space-24</code> (96px)	Hero and feature sections on corp landing
<code>--spacing-gap-tight</code>	<code>--space-2</code> (8px)	Tight component gap (icon + text, badge group)
<code>--spacing-gap-default</code>	<code>--space-4</code> (16px)	Standard component gap (card grid, form rows)
<code>--spacing-gap-loose</code>	<code>--space-8</code> (32px)	Loose layout gap (sidebar items, content blocks)

7.4 Container widths

Three containers — narrow, page, and wide — cover all known layout needs.

Token	rem	px	Use
<code>--container-narrow</code>	42rem	672px	Long-form text (ZugaLearn lesson, ZugaLife onboarding copy, legal pages). Reading width capped to prevent line lengths beyond ~75 characters.
<code>--container-page</code>	72rem	1152px	Standard page width. ZugaApp studio content, ZugaTrader dashboard, ZugaImage gallery.
<code>--container-wide</code>	90rem	1440px	Full-width marketing surfaces (corp landing sections). ZugaVideo wallpaper browser. Does not apply padding — layout components inside are responsible for gutters.

All containers are centered with `margin-inline: auto`. Padding is applied by the layout wrapper, not the container token itself.

7.5 Breakpoints

Token	Value	Behavior
<code>--breakpoint-sm</code>	640px	Single-column stacks at < sm. Minimum supported width for mobile web.
<code>--breakpoint-md</code>	768px	Two-column layouts unlock at ≥ md. Tablet portrait target.
<code>--breakpoint-lg</code>	1024px	Sidebar nav visible at ≥ lg. Desktop default.
<code>--breakpoint-xl</code>	1280px	Wide content layouts (ZugaTrader multi-column dashboard) at ≥ xl.
<code>--breakpoint-2xl</code>	1536px	Max-width clamping target. Layouts above this width use <code>--container-wide</code> centering and do not expand further.

Mobile-first convention: all media queries are min-width. Components start from the narrowest case (< 640px) and progressively unlock layout complexity. Overlay components

(ZugaGamerOverlay, ZugaTraderOverlay) are exempt — they target fixed desktop viewport sizes and do not need responsive breakpoints.

7.6 Density system

The token system exposes two density multipliers. Sub-brand profiles override `--density-*` to control spacing tightness for data-heavy studios.

Token	Value	Profile
<code>--density-comfortable</code>	1	Default. All studios unless overridden. ZugaLife, ZugaLearn, ZugaCloud, corp landing.
<code>--density-compact</code>	0.875	Data-heavy studios. ZugaTrader dashboard, ZugaCode diff/review. Apply by multiplying spacing tokens: <code>calc(var(--space-4) * var(--density-compact))</code> .

How density is applied: The multiplier is not applied globally — it is applied per-component in data-dense studio profiles. A card's padding, row height, and gap use density as a multiplier. Typography size tokens are NOT density-controlled — font sizes stay fixed regardless of density. Only spacing and layout dimensions shrink.

Comfortable is the default because most studios (wellness, creative, education) benefit from generous whitespace. Compact is an explicit opt-in for studios where information density is a feature, not a tradeoff.

7.7 Z-index stack

Z-index values are in the token system to prevent the ad-hoc integer race that breaks every long-lived UI codebase.

Token	Value	Layer
<code>--z-base</code>	0	Default flow elements
<code>--z-dropdown</code>	10	Dropdown menus, select popups
<code>--z-sticky</code>	20	Sticky headers, pinned sidebars
<code>--z-modal-backdrop</code>	30	Modal overlay backdrop
<code>--z-modal</code>	40	Modal content
<code>--z-toast</code>	50	Toast notifications (above modals)
<code>--z-tooltip</code>	60	Tooltips (above everything)

Rule: never hardcode a z-index integer in component code. Always reference a `--z-*` token. If a new layer is needed, add a token — don't increment a hardcoded value.

7.8 Shadow scale

Shadows provide elevation cues. They are subtle — the design language does not use aggressive drop shadows.

Token	Use
<code>--shadow-xs</code>	Card resting state in light mode

Token	Use
<code>--shadow-sm</code>	Input focus ring supplement, small interactive element lift
<code>--shadow-md</code>	Card on hover, dropdown panel
<code>--shadow-lg</code>	Modal, flyout panel
<code>--shadow-xl</code>	Full-page overlay, popover anchored to viewport
<code>--shadow-ring</code>	Focus ring — 3px offset ring using <code>--border-focus</code> (cyan-500)

Dark-mode note: Shadows are less perceptible on dark surfaces. In dark mode, elevation is primarily communicated through surface stops (`--surface-secondary`, `--surface-tertiary`) rather than drop shadows. Components should not rely solely on shadow for elevation communication.

7.9 Grid

No dedicated grid token exists — layout grid is handled by CSS Grid/Flexbox with gap values drawn from the semantic spacing tokens. The standard patterns:

Card grid: `display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: var(--spacing-gap-default);`

Sidebar + main: `display: grid; grid-template-columns: 240px 1fr; gap: var(--spacing-gap-loose);`

Form fields: `display: flex; flex-direction: column; gap: var(--spacing-gap-default);`

Inline group (icon + label, badge + text): `display: flex; align-items: center; gap: var(--spacing-gap-tight);`

The sidebar column width (240px) is not a token — it is a layout-specific dimension that varies by studio. ZugaTrader uses a narrower sidebar (200px) in compact density mode; ZugaApp shell uses 240px.

8. Iconography

8. Iconography

Authored: 2026-04-25 Mike checkpoints: none (Antonio-final)

All iconography decisions in this section are fully resolved. No checkpoints. Sizing references -- space-* tokens from §7. Spacing & Layout. Custom mark variants reference §2. Brand Foundation (wm02).

8.1 Primary library — Lucide

Lucide is the primary icon library across all Zuga products.

Why Lucide:

- MIT licensed** — no attribution required, safe for commercial product and open-core distribution.
- Broad coverage** — covers the full range of UI primitives needed across 14+ studios without gaps (navigation, data, media, feedback, commerce, productivity).
- Feather lineage** — Lucide is a maintained fork of Feather Icons. The visual language (open, geometric stroke icons) is well-established and familiar without being generic.
- Consistent defaults** — every icon ships at 24×24px on a 24-unit grid with a 2px stroke. No per-icon sizing negotiation.
- Framework-agnostic** — available as standalone SVG, Vanilla JS, React, Vue, and Svelte packages. Matches the Zuga multi-framework reality (Vue in studios, Astro on corp landing, Electron for overlay).

When sourcing icons, use the latest stable Lucide release pinned in the consuming repo’s package.json. Do not vendor SVG copies unless the icon is a custom Zuga mark (see §8.4).

8.2 Grid and stroke

Property	Value	Rationale
Grid	24×24px	Lucide default. Aligns with --space-6 (1.5rem / 24px) at base density.
Stroke weight	2px	Matches body text weight. Inter/system-sans rendered at 16-18px medium weight reads as a similar visual mass to a 2px stroke outline at 24px. Thinner strokes (1px, 1.5px) feel anemic at this size. Thicker strokes (2.5px, 3px) read as heavy and misalign with the type weight.
Line cap	Round	Lucide default. Round caps read softer — consistent with the brand’s non-aggressive tone.
Corner join	Round	Lucide default.
Fill	None (outline only)	Filled variants are reserved for active/selected state indicators only — an icon in its default state is always outline.

Filled / active variant rule: When a filled version of an icon is needed to communicate a selected or active state (e.g., a bookmarked item, an active nav item), use the Lucide `*-fill` suffix variant where available, or apply `fill: currentColor; stroke: none` manually. Do not use filled icons as the default resting state.

8.3 Sizing tokens

Icon sizing maps to the `--space-*` scale established in §7. Spacing & Layout. Never hardcode icon dimensions — reference space tokens.

Size class	Space tokens	px range	Typical use
icon-xs	<code>--space-3</code> to <code>space-4</code>	12–16px	Badge pips, inline label decorators, dense data tables, compact toolbar
icon-sm	<code>--space-4</code> to <code>space-5</code>	16–20px	Secondary actions, list item decorators, breadcrumb separators
icon-md	<code>--space-5</code> to <code>space-6</code>	20–24px	Default. Primary action buttons, nav items, card CTAs. The 24px end maps to the Lucide native grid.
icon-lg	<code>--space-8</code> to <code>space-10</code>	32–40px	Empty state illustrations, feature callouts, marketing icon grids

The native Lucide SVG is 24×24. Rendering at `icon-xs` or `icon-sm` requires explicit width/height overrides on the `<svg>` element or a wrapping container — do not rely on CSS `font-size` scaling for SVG icons in production.

Touch target minimum: Icon-only interactive elements must have a minimum touch target of 44×44px regardless of the rendered icon size. Apply padding to the button/anchor wrapper, not the icon itself.

Icon-only buttons require ARIA labels. An `<button>` containing only an icon with no visible text label must have `aria-label` or `aria-labelledby`. See §13. Accessibility (cross-reference when authored).

8.4 Custom Zuga marks

Three categories of custom marks exist alongside the Lucide library.

Z monogram and logo mark variants

The Z monogram and logo mark glyph are defined in §2. Brand Foundation (wm02 checkpoint). These marks are not sourced from Lucide — they are produced by the brand design process and versioned as SVG assets in `packages/design-tokens/assets/` (path to be established when wm02 is finalized). Use only the canonical exported SVG; do not redraw from memory or approximate.

Z monogram variants by context:

Variant	Use
Monochrome light	App icons, favicons, avatar slots on dark backgrounds
Monochrome dark	Avatar slots on light backgrounds, watermarks
Cyan-tinted	Product hero sections, marketing surfaces where brand color is the intent
Lockup (wordmark + glyph)	Full logo — header nav, email headers, social card OG images

ZugaShield

ZugaShield is the custom security/trust mark used in ZugaID and authentication surfaces. It is not a Lucide icon. Treat it as a brand asset with the same versioning rules as the Z monogram.

Sub-brand glyphs

Studios that have distinct domain icons (e.g., ZugaTrader chart mark, ZugaGamer overlay icon) maintain their own glyph assets. These are allowed to diverge from Lucide’s visual language for domain authenticity (a trading chart icon needs precision; a music waveform icon benefits from custom curves), but must conform to the 24×24 grid and 2px stroke weight to remain composable with Lucide icons in mixed contexts.

8.5 Usage rules

Render icons at their token-specified size. Do not scale icons with `transform: scale()` in production — it blurs subpixel rendering on non-retina displays.

Icon color inherits `currentColor` by default. Do not hardcode fill or stroke colors on icons. Set color on the parent element.

Do not mix icon libraries. If Lucide has a close enough version of what you need, use it. Introducing a second icon library (Heroicons, Phosphor, etc.) for one-off icons creates visual inconsistency across studios. The only exception is custom Zuga marks (§8.4).

No emoji as icons. Emoji rendering is platform-dependent, inconsistent across OS and browser, and cannot be styled with CSS. Use Lucide icons for all UI icon needs.

No rasterized icons. All icons in UI are SVG. PNG/WebP icon sprites are not used.

9. Imagery & Illustration

9. Imagery & Illustration

Authored: 2026-04-25 Mike checkpoints: none (Antonio-final)

This section sets sourcing and treatment rules for photography and illustration. These are design constraints that apply when anyone on the team — engineer, marketer, or contractor — is choosing or producing imagery for a Zuga surface.

9.1 Photography direction

Mood: Engineered, considered, AGI-future.

Translated into rules:

Real production over stock. Prefer screenshots and recordings of actual Zuga products — real ZugaApp dashboards, real ZugaTrader charts mid-session, real ZugaLife habit completions. These images are self-documenting and brand-authentic. Stock photos of “technology” are not.

Dark surfaces preferred for dark-mode-first products. When showcasing a Zuga product on marketing surfaces, use dark-mode screenshots unless the studio is explicitly light-mode (ZugaLife wellness surfaces are acceptable either way). Dark backgrounds make the cyan and emerald accent colors read at full saturation.

Minimal post-processing. No heavy color grading, no lens flares, no Instagram-filter warmth. Light adjustments for brightness and contrast are acceptable. The goal is “what it actually looks like to use this,” not “what we wish it looked like.”

Environment context, not people. When context shots are needed, show the environment where the product is used (a trading setup, a code editor, a game stream) rather than staged photos of people interacting with it. If a person must appear, they should be doing something real, not posed.

Technical precision over lifestyle aspiration. Charts should be legible. Code should be real. UI states should be accurate. A screenshot of a ZugaTrader signal firing is more credible than a photo of a phone floating in front of a sunset.

Avoid imagery that ages. Hardware, fashion, and desktop wallpapers date screenshots. Use neutral environments. Blur or crop out incidental third-party branding where possible.

9.2 Illustration style

Illustration is geometric, abstract, and uses a limited palette drawn from the token system.

Style rules:

Geometric, not organic. Shapes are rectangles, circles, lines, and their combinations. No hand-drawn or loose-stroke illustration. The visual vocabulary is system-level, not artisanal.

Abstract over literal. An illustration representing ZugaTrader does not need to be a picture of a chart. It can be a grid of lines that implies data flow. Abstraction ages better and travels across cultural contexts without misreading.

Limited palette. Master Zuga palette for illustration: `--color-cyan-500` / `--color-cyan-400` (primary), `--color-slate-800` / `--color-slate-700` (background mass), `--color-slate-300` (secondary structural elements). Sub-brand accent colors are allowed when the illustration is scoped to one studio. Do not introduce colors outside the token system.

Single-color preferred for small contexts. At icon-lg size and below, use single-color (monochrome). Multi-color illustration is reserved for hero sections and feature callouts at larger sizes.

No gradients from outside the palette. If a gradient is used (e.g., a hero background fill), it must be composed from stops within the token color ramps — not from arbitrary interpolated hues.

9.3 Hero patterns

Hero sections (corp landing, studio feature pages) use geometric background patterns to add depth without photographic complexity.

Acceptable hero pattern treatments:

Grid overlays: A fine-line grid (1px `--color-slate-700` at 8–12% opacity) on `--surface-inverse` communicates precision and structure. Common in ZugaTrader and ZugaCode hero sections.

Dot fields: Regularly-spaced dot grids (same opacity treatment) work for more open, spacious contexts like ZugaLife and ZugaCloud.

Diagonal line fields: Sparse hatching at 30-45° adds texture to marketing surfaces without weight. Use `--color-slate-600` at 10-15% opacity.

What patterns are not:

Not brand illustrations. A hero pattern is a background texture — it does not carry narrative.

Not full-bleed photography replacements. They accompany UI screenshots or illustration; they don't stand alone as a hero "image."

Hero patterns are Phase 1 — elaborate-on-need. The corp landing site currently uses a combination of grid overlay and feature product screenshots. That combination is the working baseline.

9.4 Do / Don't

Do

Do use real ZugaApp, ZugaTrader, or ZugaLife product screenshots on marketing surfaces.

Do show UI in the actual state a user would see it — data visible, not empty states.

Do use dark-mode product screenshots on dark marketing backgrounds.

Do apply minimal exposure/contrast correction to screenshots for web (brighten highlights slightly for readability on marketing surfaces).

Do use geometric illustration at icon-lg size for empty states and feature callouts.

Do use the corp landing geometric grid pattern as a hero background texture.

Do source custom photos of actual hardware setups (trading station, code workstation) when environment context is needed.

Don't

Don't use stock photography of people at computers, at whiteboards, or in "team collaboration" poses — especially the trope of strangers laughing at laptops.

Don't use imagery that implies features or UI states that aren't shipped — no Figma mockups of unreleased flows, no screenshots annotated with speculative data.

Don't use imagery sourced from generic "AI / technology" stock libraries (glowing neural networks, floating holographic interfaces, futuristic cityscapes).

Don't use screenshots from competitor products — even as "context" or "comparison."

Don't use UI mockups (Figma frames, browser chrome templates) in place of real product screenshots. If the product isn't built yet, use illustration or describe it in text.

Don't apply heavy photo filters, dramatic vignettes, or color grading that changes the perceived color of the Zuga UI.

Don't use imagery that shows outdated versions of the UI — keep screenshots current to the deployed product.

10. Motion

10. Motion

Authored: 2026-04-25 Mike checkpoints: none (Antonio-final)

Motion in Zuga products is functional, not decorative. Every transition either communicates state change, guides spatial orientation, or confirms user action. Motion that does none of these things is removed. All values below are sourced from `packages/design-tokens/src/tokens.css` — do not hardcode durations or easing functions in component code.

10.1 Duration tokens

Token	Value	Use
--motion-duration-instant	75ms	Immediate feedback. Tooltip appear/disappear, active/pressed state, toggle switch snap.
--motion-duration-fast	150ms	Short UI transitions. Button hover state, checkbox check, badge update, inline input focus ring.
--motion-duration-normal	200ms	Default transition duration. Most component enter/exit animations. Dropdown open, modal fade-in, tab switch.
--motion-duration-slow	300ms	Larger layout changes. Sidebar collapse/expand, sheet slide-in, page-level route transitions where elements travel a large distance.
--motion-duration-slower	500ms	Deliberate, weighty transitions. Full-page entry animation, complex choreography where multiple elements move in sequence, onboarding reveal sequences.

Rule: exit is faster than enter. When a component enters, use its assigned duration. When it exits, use one step shorter. A modal opens at `--motion-duration-normal` (200ms) and closes at `--motion-duration-fast` (150ms). This makes the interface feel responsive to dismissal — users should never feel they are waiting for something to disappear.

10.2 Easing tokens

Token	Value	Character
--motion-easing-standard	cubic-bezier(0.2, 0, 0, 1)	Smooth start, crisp arrival. The default for most UI transitions.
--motion-easing-emphasized	cubic-bezier(0.3, 0, 0, 1)	Slightly more deliberate start than standard. Use for important reveals — modal entrance, hero section load, onboarding step.

Token	Value	Character
--motion-easing-decel	cubic-bezier(0, 0, 1)	Zero initial velocity, decelerates to stop. Entering elements. An element dropping into the viewport from above, a drawer sliding in — it starts fast and settles.
--motion-easing-accel	cubic-bezier(0.3, 0, 1, 1)	Starts slowly, accelerates out. Exiting elements. An element dismissed should accelerate away, not drift off.

Pairing rule: Entering elements use --motion-easing-decel; exiting elements use --motion-easing-accel. Elements that stay in place and change state (color, scale, opacity in-place) use --motion-easing-standard. High-stakes reveals use --motion-easing-emphasized.

Do not use spring/elastic curves for UI control transitions. Bounce and elastic easing are appropriate for celebratory states — a success confirmation, a streak badge unlock, a goal completion animation. They are not appropriate for click/hover feedback, dropdown open/close, or modal transitions. Bounce on interactive controls makes the interface feel playful at the cost of user time — every overshoot adds perceived latency to an action the user has already committed to.

10.3 Easing in practice

Quick reference for common component patterns:

Pattern	Duration	Easing
Button hover	--motion-duration-fast	--motion-easing-standard
Button active/pressed	--motion-duration-instant	--motion-easing-standard
Dropdown open	--motion-duration-normal	--motion-easing-decel
Dropdown close	--motion-duration-fast	--motion-easing-accel
Modal open	--motion-duration-normal	--motion-easing-emphasized
Modal close	--motion-duration-fast	--motion-easing-accel
Toast enter	--motion-duration-normal	--motion-easing-decel
Toast exit	--motion-duration-fast	--motion-easing-accel
Sidebar collapse/expand	--motion-duration-slow	--motion-easing-standard
Page route transition	--motion-duration-slow	--motion-easing-emphasized
Tab switch	--motion-duration-normal	--motion-easing-standard
Success / badge celebrate	--motion-duration-slower	Spring/elastic allowed
Tooltip show	--motion-duration-instant	--motion-easing-standard

10.4 Choreography principles

Parent-first reveal

When a container and its children both animate on entry, the parent reveals first. Children animate after the parent's boundary is established, staggered by --motion-duration-instant (75ms)

increments. The stagger maximum is `--motion-duration-fast` (150ms) — after that, the user is waiting, not perceiving sequence.

Why: If children appear before the parent container, the result is elements floating in space before context arrives. The parent establishes the spatial reference frame; children populate it.

Testable rule: Visually inspect any entry animation. If content appears in a region before that region's boundary is visible, the animation order is wrong.

Exit faster than enter

Enter and exit durations are not symmetric. Exit uses one duration step shorter than enter. (If enter uses `normal`, exit uses `fast`. If enter uses `slow`, exit uses `normal`.)

Why: After a user dismisses something, the cognitive task is done. The animation plays out for the user's perceptual system to process the state change — but prolonged exit animations feel like UI holding the user in place after they've moved on. Fast exits confirm the action; they don't delay it.

Testable rule: In any component with paired enter/exit animation, measure both durations. Exit must be shorter. Equal durations are a defect; exit longer than enter is a bug.

No bounce on interactions

Hover, click, focus, toggle, and drag interactions do not use spring or elastic easing. Control feedback must complete without overshoot. The only exceptions are:

Celebratory states: success confirmation animations, achievement unlocks, streak completions. These communicate "something positive happened" — a single bounce or elastic settle is appropriate and expected.

Loading/progress indicators: a pulsing or bouncing animation on a loading spinner is acceptable because the user is waiting, not acting.

Why: Elastic easing on interactive controls adds frames after the motion should be complete. Every overshoot frame is a moment where the UI looks like it's catching up to the user's input. On fast machines this is barely perceptible; on slower hardware it compounds with rendering lag and makes the interface feel sluggish.

10.5 prefers-reduced-motion contract

Zuga products honor the `prefers-reduced-motion`: `reduce` media query. This is not optional.

Contract:

When `prefers-reduced-motion: reduce` is active, all transition and animation durations collapse to `--motion-duration-instant` (75ms) or are removed entirely.

Easing simplifies to `linear` — no acceleration or deceleration curve processing.

Parallax, scroll-driven animations, and auto-playing motion are disabled entirely (not just slowed down).

State changes still communicate — they happen instantly rather than not at all. A modal still "appears"; it just does so without a fade.

Implementation baseline:

```
@media (prefers-reduced-motion: reduce) {  
  *, *::before, *::after {  
    animation-duration: 0.01ms !important;  
    transition-duration: 0.01ms !important;  
  }  
}
```

This is a global reset baseline. Individual components should also check `prefers-reduced-motion` at the component level when they have complex choreography (stagger sequences, scroll-driven reveals) that the global reset does not fully suppress.

Carve-out for essential motion: animation that communicates ongoing system activity (spinners, progress bars, skeleton-loader pulses) should be *reduced in amplitude*, not killed. A loading spinner that runs at slower speed or a progress bar that updates without a smooth fill is acceptable; an invisible indicator that leaves the user staring at a frozen UI is not. Components implementing essential motion should check `prefers-reduced-motion` themselves and override the global rule with a reduced-amplitude alternative.

Note: `prefers-reduced-motion: reduce` does not mean “no motion.” It means “less motion” — prioritize respecting the user’s system preference over exact animation parity. The full accessibility rationale and WCAG mapping lives in §13. Accessibility (cross-reference when authored in T27). This section states the implementation contract; §13 states the rationale.

10.6 What not to animate

Some transitions should not be animated regardless of duration or easing:

Color theme switches (light/dark mode toggle). Animating a color-scheme switch causes the entire page surface to flash through intermediate states. Snap instantly.

Skeleton screen replacements. When a skeleton loader resolves to real content, replace instantly. Fading from skeleton to content draws attention to the loading artifact rather than the content.

Error state appearances. Errors should demand immediate attention — fade-in softens the signal. Render error states at full opacity without transition.

Focus ring appearance. Focus indicators must appear without delay. A focus ring that fades in is a WCAG violation for keyboard-only users who depend on it for spatial orientation.

11. Components

11. Components

Authored: 2026-04-25 (Task 25)

This section documents the 13 UI components that form the Zuga component layer. Each component page covers anatomy, variants, accessibility (keyboard, ARIA, contrast, focus), code examples in Astro + Vue + plain HTML, a Don'ts table, and a full token reference. All component tokens are Tier 3 (scoped, --component-*), compose from Tier 2 semantic tokens, and are listed in packages/design-tokens/src/components/.

Component index

Component	Summary
Avatar	Circular identity element representing a user or entity
Badge	Small label for status, count, or categorization
Button	Primary interactive trigger for actions
Card	Surface container for grouping related content or actions
Checkbox	Binary selection control for forms and settings
Input	Single-line text entry field
Modal	Overlay dialog for focused content or decisions
Nav	Navigation component for studio-level wayfinding
Select	Dropdown for selecting a single option from a set
Skeleton	Animated loading placeholder
Tabs	Segmented panel switcher within a view
Toast	Ephemeral notification for confirmations and alerts
Toolbar	Horizontal strip of context-sensitive controls

12. Patterns

12. Patterns

Authored: 2026-04-25 Authoring task: see plan Task 26

[draft content — see plan Task 26]

13. Accessibility

13. Accessibility

Authored: 2026-04-26 Authoring task: see plan Task 27

Zuga targets **WCAG 2.2 AA** across all products. This section is the governance layer — it states thresholds and rules. Per-component measured contrast ratios and ARIA implementation details live in §11 component pages (button, modal, input, etc.). Cross-references to §10.5 (reduced-motion carve-outs) are noted where relevant.

13.1 Conformance target

Level	Target	Notes
WCAG 2.2 AA	Required	All shipped components and patterns
WCAG 2.2 A	Required	Baseline subset of AA
WCAG 2.2 AAA	Aspirational	Target Size Enhanced (2.5.5) noted below; not a blocking gate
Non-conformance on any AA criterion is a ship blocker for the affected component.		

13.2 Color and contrast — per-role minimums

These are governance thresholds, not measured values. Measured ratios for specific token pairs are documented in each §11 component page.

Role	Criterion	Threshold	Notes
Body text (≤18px or ≤14px bold)	1.4.3 AA	4.5 : 1	Against background surface
Large text (≥18px regular or ≥14px bold)	1.4.3 AA	3 : 1	Against background surface
UI components (focus rings, borders, icons)	1.4.11 AA	3 : 1	Against adjacent color
Graphical objects (chart lines, SVG icons)	1.4.11 AA	3 : 1	Against adjacent color
Feedback text (success, warn, danger)	1.4.3 AA	4.5 : 1	Body-size feedback labels
Disabled state	1.4.3 AA	exempt	Disabled elements are explicitly exempt; must still be perceivable
Phase 2 aspirational (body)	1.4.6 AAA	7 : 1	Not required; target for high-stakes surfaces

The `--accent-brand` default resolves to `--color-cyan-700` (#0e7490), which passes 4.5:1 against `--accent-fg` (ffffff). Sub-brand profiles that override `--accent-brand` are responsible for maintaining compliance at the same threshold.

13.3 Color-only signaling — banned

WCAG 1.4.1 (Use of Color) is non-negotiable. Color alone must never be the sole means of conveying information, indicating an action, prompting a response, or distinguishing a visual element.

Practical rules:

Error states: icon + text label required alongside any red color treatment. Red border alone fails.

Success/warn/danger toasts: must include an icon (check, warning triangle, X) in addition to feedback color.

Form field validation: invalid state requires visible text error message (→ WCAG 3.3.1). Red outline alone fails.

Charts and graphs: data series must be distinguishable by shape, pattern, or label, not color alone.

Link detection: links within body text must have a non-color visual indicator (underline or weight change) unless the contrast ratio between the link and surrounding text is $\geq 3:1$.

When in doubt: remove all color from the UI in your head and ask whether the information still comes through. If not, it fails 1.4.1.

13.4 Keyboard model

Every interactive element must be fully operable by keyboard alone (WCAG 2.1.1). There are no exceptions for components where mouse interaction is “standard” (drag-to-reorder, date pickers, sliders — all need keyboard equivalents).

Navigation keys

Key	Action
Tab	Advance focus to next focusable element
Shift+Tab	Reverse focus to previous focusable element
Enter	Activate: buttons, links, menu items, checkboxes
Space	Activate: buttons, checkboxes; scroll (when not trapped)
Esc	Close: modals, dialogs, tooltips, popovers, dropdowns
Arrow keys	Navigate within a widget: tabs, radio groups, menus, listboxes
Home / End	Jump to first / last item in a list or text field

Focus trap rules

Modals trap focus by design — this is correct behavior, not a keyboard trap (WCAG 2.1.2). A modal focus trap is allowed because pressing Esc always exits, making focus escapable. Focus must never be trapped in a way that requires a mouse to escape.

Focus order must match visual and reading order (WCAG 2.4.3). If CSS order differs from DOM order, DOM order wins for keyboard navigation.

Focus visibility

Focus indicators must always be visible (WCAG 2.4.7). The Zuga focus indicator is --shadow-ring (3px cyan ring via box-shadow). This ring must not be suppressed via `outline: none` without a

replacement indicator of equal or greater visibility. Focused elements must not be fully obscured by sticky headers, banners, or overlays (WCAG 2.4.11).

13.5 Target size

Standard	Criterion	Minimum size	Gate
Zuga minimum	2.5.8 AA	24 × 24 px	Ship blocker
Zuga aspirational	2.5.5 AAA	44 × 44 px	Best-effort; applied to primary actions
The 24×24px minimum applies to the interactive click/touch target, not the visual icon. Icon-only buttons with 16px icons must have padding that brings the total target to at least 24×24px.			

13.6 Screen reader patterns

Semantic HTML first

Native HTML elements carry implicit roles and states. Prefer `<button>` over `<div role="button">`, `<nav>` over `<div role="navigation">`, `<input type="checkbox">` over `<div role="checkbox">`. Only add ARIA when native semantics are insufficient.

Required aria-* defaults by primitive

Primitive	Required ARIA	Notes
Icon-only button	<code>aria-label="[action]"</code>	Visible label text alone is insufficient for icon-only; label must describe the action
Toggle button	<code>aria-pressed="true"</code>	<code>false"</code>
Loading skeleton	<code>aria-busy="true"</code> on parent container	Announce when content is loading
Toast / alert	<code>role="status" or role="alert" + `aria-live="polite`</code>	<code>assertive"</code>
Modal / dialog	<code>role="dialog" + aria-modal="true" + aria-labelledby</code> by pointing to heading	Required; focus trap applies
Tooltip	<code>role="tooltip" + id + trigger</code> <code>aria-describedby</code> by pointing to tooltip id	Only for supplemental information; don't hide critical info in tooltips
Progress / spinner	<code>role="progressbar" + aria-valuenow + aria-valuemin + aria-valuemax</code> <code><label></code> associated via <code>for/id</code> or <code>aria-label</code> ;	Use <code>aria-label</code> if no visible label
Form input	error state via <code>aria-describedby</code> by pointing to error element	Never label via placeholder alone
Navigation landmark	<code><nav aria-label="[unique name]"></code>	Multiple <code><nav></code> elements need distinct labels
Collapsible / accordion	trigger: <code>aria-expanded="true"</code>	<code>false" + aria-controls`</code> pointing to panel id

13.6.1 Name, Role, Value (WCAG 4.1.2)

Every interactive element must have:

Accessible name — provided by `<label>`, `aria-label`, or `aria-labelledby`.

Role — provided by native HTML element or explicit role attribute.

State/value — dynamic state changes (`aria-checked`, `aria-expanded`, `aria-disabled`) must be kept in sync with visual state via JS.

13.6.2 Status messages (WCAG 4.1.3)

Non-modal status messages (toasts, form save confirmations, inline validation success) must be announced to screen readers without moving focus. Use `aria-live` regions:

`aria-live="polite"` — normal status messages. Screen reader finishes current utterance first. Use for toasts, form save confirmations.

`aria-live="assertive"` — urgent interruptions only (unrecoverable errors, session expiry). Interrupts screen reader immediately.

The live region element must exist in the DOM before content is injected into it. Injecting a live region and content simultaneously does not reliably trigger announcement.

13.7 Error handling accessibility

Rule	Criterion
Errors must be described in text, not just color or icon	3.3.1
When the cause of an error is known, suggest the correction	3.3.3
Legal, financial, or data-deleting actions require confirmation, undo, or review step	3.3.4
Don't ask users to re-enter information they've already provided in the same session	3.3.7
Auth flows must not require cognitive function tests (CAPTCHA) without an accessible alternative	3.3.8

13.8 Text spacing (WCAG 1.4.12)

Content must not lose information or functionality when the user overrides text spacing to:

Line height $\geq 1.5 \times$ the font size

Paragraph spacing $\geq 2 \times$ the font size

Letter spacing $\geq 0.12 \times$ the font size

Word spacing $\geq 0.16 \times$ the font size

The Zuga default tokens already satisfy these thresholds. The rule is that custom components must not hard-code heights or clamp containers in ways that clip text when spacing is increased.

13.9 Tooltips and hover content (WCAG 1.4.13)

Tooltips and popovers triggered by hover or focus must be:

Dismissible — user can dismiss without moving pointer or focus (typically Esc).

Hoverable — pointer can move over the tooltip without the tooltip disappearing.

Persistent — tooltip stays visible until dismissed, focus moves away, or hover leaves the trigger + tooltip area.

13.10 Reduced motion

Zuga's motion system includes a `prefers-reduced-motion` layer. See §10.5 for the full essential-motion carve-out policy.

Summary of §13 rules:

All transitions and animations must be gated by `@media (prefers-reduced-motion: reduce)`. "Essential motion" (progress spinners, loading states that convey real-time information) may retain reduced but not eliminated animation.

Position-changing transforms (slide-ins, parallax, scroll-jacking) must stop completely under reduced-motion preference.

The `--motion-duration-*` tokens are used throughout components. Reduced-motion overrides set these to `0ms` or near-zero on the component scope. Never hard-code duration values in component CSS — always reference the tokens so the override layer works.

13.11 Component-level deep dives

Per-component accessibility specifics (measured contrast ratios, exact ARIA patterns, keyboard interaction models) live in the §11 component pages:

`docs/bible/11-components/button.md` — button, icon-button, toggle patterns

`docs/bible/11-components/input.md` — text input, error association

`docs/bible/11-components/modal.md` — focus trap, dialog ARIA, Esc behavior

`docs/bible/11-components/toast.md` — live region implementation

`docs/bible/11-components/checkbox.md` — `aria-checked`, indeterminate state

`docs/bible/11-components/select.md` — listbox pattern, keyboard navigation

`docs/bible/11-components/tabs.md` — tab/tabpanel ARIA, arrow-key navigation

`docs/bible/11-components/nav.md` — landmark labels, active state

14. Code Tokens

14. Code Tokens

Authored: 2026-04-26 Authoring task: see plan Task 27

This section is the engineer’s entry point to the token system. For override rules (which tokens a sub-brand profile can change), see §15. For cross-platform delivery beyond web CSS, see §16.

14.1 Package

```
@zuga-technologies/design-tokens v0.1.0
```

Published to the GitHub Package Registry (npm.pkg.github.com). Access is restricted — you need a GitHub PAT with `read:packages` scope in your `.npmrc`.

```
// .npmrc
```

```
@zuga-technologies:registry=https://npm.pkg.github.com
```

Exports

Export path	What it is
@zuga-technologies/design-tokens/tokens.css	Tier 1 primitives + Tier 2 semantic defaults (light mode)
@zuga-technologies/design-tokens/tokens-dark.css	Dark mode semantic overrides
@zuga-technologies/design-tokens/reset.css	Zuga CSS reset (box-sizing, margin zeroing)
@zuga-technologies/design-tokens/profiles/*.css	Sub-brand profile overrides (e.g., <code>profiles/studio-life.css</code>)
@zuga-technologies/design-tokens/components/*.css	Component-scoped Tier 3 tokens

14.2 Naming convention (DTCG-aligned)

Tokens follow a three-tier hierarchy. The CSS custom property name encodes the tier.

```
--{tier-prefix}-{category}-{variant}
```

Tier 1 — Primitives

Raw values. No semantic meaning. Never referenced directly in component code — use Tier 2 instead.

```
--color-{ramp}-{stop}      e.g. --color-cyan-500
--space-{n}                 e.g. --space-4
--radius-{name}             e.g. --radius-md
--shadow-{name}              e.g. --shadow-ring
--motion-duration-{name}    e.g. --motion-duration-fast
```

```
--motion-easing-{name}      e.g. --motion-easing-standard
--z-{name}                  e.g. --z-modal
--container-{name}          e.g. --container-prose
--breakpoint-{name}         e.g. --breakpoint-lg
--type-scale-{name}         e.g. --type-scale-base
--font-family-{name}        e.g. --font-family-sans
--font-weight-{name}        e.g. --font-weight-semibold
--font-line-height-{name}   e.g. --font-line-height-normal
```

Tier 2 — Semantic

Carry meaning. Resolved from Tier 1 values. These are what components use.

```
--surface-{name}           e.g. --surface-primary
--text-{name}               e.g. --text-primary
--border-{name}             e.g. --border-focus
--accent-{role}             e.g. --accent-brand
--feedback-{type}           e.g. --feedback-success
--density-{mode}            e.g. --density-compact
```

Tier 2 is split into **locked** and **overridable**:

surface.*, text.*, border.*, feedback.* — **locked**. No profile override.

accent.*, density.* — **overridable** by sub-brand profiles (see §15).

Tier 3 — Component

Scoped to a single component. Declared inside component CSS files.

```
--component-{name}-{property}  e.g. --component-button-radius
```

Component tokens are overridable per-profile. Tier 1 and locked Tier 2 tokens are not.

14.3 Override rules summary

Only three prefix groups are overridable by sub-brand profiles:

--accent-*

--density-*

--component-*

Everything else is locked. The lint enforcer for this rule lives at packages/design-

tokens/scripts/tokens-lint.mjs. See §15 for the full enforcement model and the profile registration gate.

14.4 Code examples

Astro

Import the package CSS in your global stylesheet. Token variables are then available everywhere.

```
/* src/styles/global.css */
@import "@zuga-technologies/design-tokens/tokens.css";
```

```

@import "@zuga-technologies/design-tokens/tokens-dark.css";
@import "@zuga-technologies/design-tokens/reset.css";

/* If this consumer uses a sub-brand profile: */
@import "@zuga-technologies/design-tokens/profiles/studio-life.css";

```

Reference tokens in component styles:

```

---
// src/components/HeroCard.astro
---
<div class="hero-card">
  <h2>Title</h2>
  <p>Body copy</p>
</div>

<style>
.hero-card {
  background-color: var(--surface-primary);
  color: var(--text-primary);
  padding: var(--space-6);
  border-radius: var(--radius-lg);
  border: 1px solid var(--border-subtle);
}

.hero-card h2 {
  font-size: var(--type-scale-2xl);
  font-weight: var(--font-weight-semibold);
  color: var(--text-primary);
}

.hero-card p {
  font-size: var(--type-scale-base);
  color: var(--text-secondary);
  margin-top: var(--space-2);
}
</style>

```

Vue SFC

Same CSS variables — Vue scoped styles work identically to Astro component styles.

```
<!-- src/components/StatusBadge.vue -->
<template>
  <span class="badge" :class="`badge--${variant}`">
    <slot />
  </span>
</template>

<script setup lang="ts">
defineProps<{ variant: 'success' | 'warn' | 'danger' }>()
</script>

<style scoped>
.badge {
  display: inline-flex;
  align-items: center;
  gap: var(--space-1);
  padding: var(--space-0_5) var(--space-2);
  border-radius: var(--radius-full);
  font-size: var(--type-scale-sm);
  font-weight: var(--font-weight-medium);
}

.badge--success {
  background-color: var(--feedback-success-subtle);
  color: var(--feedback-success);
}

.badge--warn {
  background-color: var(--feedback-warn-subtle);
  color: var(--feedback-warn);
}

.badge--danger {
  background-color: var(--feedback-danger-subtle);
  color: var(--feedback-danger);
}
</style>
```

Plain HTML

Link the token CSS directly and use variables inline or in a <style> block.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <link rel="stylesheet" href="/node_modules/@zuga-technologies/design-tokens/src
  <style>
    body {
      background-color: var(--surface-canvas);
      color: var(--text-primary);
      font-family: var(--font-family-sans);
    }

    .cta-button {
      background-color: var(--accent-brand);
      color: var(--accent-fg);
      padding: var(--space-3) var(--space-6);
      border-radius: var(--radius-md);
      border: none;
      font-weight: var(--font-weight-semibold);
    }
  </style>
</head>
<body>
  <button class="cta-button" style="color: var(--accent-fg)">
    Get started
  </button>
</body>
</html>
```

14.5 Dark mode

Dark mode is a semantic-layer swap — Tier 1 primitives don't change, only Tier 2 surface.* and text.* values flip.

```
/* Apply dark mode at the OS level */
@import "@zuga-technologies/design-tokens/tokens.css";
@import "@zuga-technologies/design-tokens/tokens-dark.css";
```

`tokens-dark.css` overrides `--surface-*` and `--text-*` inside `@media (prefers-color-scheme: dark)`. If you need manual dark-mode toggle, apply a `[data-theme="dark"]` attribute and scope the dark overrides to that selector — do not maintain a separate stylesheet; patch `tokens-dark.css` to support both the media query and the attribute selector.

15. Sub-brand Contract

15. Sub-brand Contract

Authored: 2026-04-26 Authoring task: see plan Task 27

The sub-brand contract defines exactly what a studio or product is allowed to change about the token system — and enforces it mechanically. The goal is expressive flexibility within a bounded space. “Flexibility” means studios have a distinct personality. “Bounded” means the base system never drifts into incoherence.

Governance and RACI for this contract live in §17. New-component upstream flow is at the bottom of this section.

15.1 Override whitelist

Only three token prefix groups are overridable in a profile CSS file:

Prefix	What it covers	Example
--accent-*	Brand hue, subtle tint, strong shade, foreground	--accent-brand, --accent-brand-subtle
--density-*	Comfortable / compact density multipliers	--density-compact
--component-*	Per-component property overrides	--component-button-radius

Everything else is **locked**:

--color-*, --space-*, --radius-*, --shadow-*, --motion-*, --z-*, --container-*, --breakpoint-*, --type-*, --font-* (Tier 1) — locked. No profile may redefine any Tier 1 primitive. The full set is encoded verbatim in the TIER1_PREFIXES constant of packages/design-tokens/scripts/tokens-lint.mjs.

--surface-*, --text-*, --border-*, --feedback-* (locked Tier 2) — no profile may override these. The semantic neutrals are a shared floor all studios stand on.

The rationale: accent identity is a studio’s primary personality knob. Density is a layout knob. Component radius and spacing are finishing-touch knobs. The text and surface system is the coherence layer — if studios could override --text-primary freely, the bible’s contrast guarantees would be unenforceable.

15.2 The six enforcement mechanisms

1. Whitelist of overridable prefixes (encoded in tokens-lint.mjs)

The lint tool at packages/design-tokens/scripts/tokens-lint.mjs parses every CSS declaration in a profile file. If a variable name falls under a locked category, the lint emits a violation and exits with code 1. The categories are defined in the constants at the top of the file:

```
const TIER1_PREFIXES = ['color', 'space', 'radius', 'shadow', 'motion', 'z', 'co
const TIER2_LOCKED_PREFIXES = ['surface', 'text', 'border', 'feedback'];
const TIER2_OVERRIDABLE_PREFIXES = ['accent', 'density'];
const TIER3_PREFIX = 'component';
```

Anything not on TIER2_OVERRIDABLE_PREFIXES or TIER3_PREFIX is rejected.

Additionally, every overridable declaration must be preceded by a `/* WHY: ... */` comment with ≥ 20 characters of substantive rationale (no “TBD”, “TODO”, or “placeholder” text). This creates a human-readable audit trail inside the CSS itself.

2. tokens-lint.mjs runs on every PR via CI

The lint runs as a `pnpm lint` script on the `@zuga-technologies/design-tokens` package and as a consumer check via `zuga-tokens-lint` binary. T34 wires CI; the lint is available today as `pnpm -filter @zuga-technologies/design-tokens lint`. Any PR that touches a profile file must pass lint before merge.

3. Profile schema validation via Astro Zod (`src/content.config.ts`)

Every profile must have a companion documentation file at

`docs/bible/profiles/<profile>.md`. The profiles content collection enforces this schema:

```
schema: z.object({
  profile: z.string(),
  consumer: z.string(),
  family: z.enum(['master', 'wellness', 'markets', 'creative', 'dev', 'cloud', '
  accentRamp: z.string(),
  rationale: z.string().min(40),
  deviations: z.array(z.string()).default([]),
})
```

`astro check` will fail if a profile doc is missing or has an invalid schema. This ties documentation to implementation — you cannot ship a new profile without the doc.

4. consuming-repos.json registry

Every consumer of `@zuga-technologies/design-tokens` must register itself in `consuming-repos.json` at the package root. The registry maps consumer repos to their profile choice. The lint tool's `lintConsumerDir()` function checks that the profile named in a consumer's `.zuga-design.json` manifest actually exists in the profiles directory. Unregistered consumers or consumers pointing at nonexistent profiles fail lint.

5. drift-detect.mjs nightly GH Action

`packages/design-tokens/scripts/drift-detect.mjs` runs nightly via GitHub Actions (T19 implementation). It checks that every registered consumer is using the current published version of

@zuga-technologies/design-tokens and that no consumer has imported token CSS from outside the package (copy-pasting a token file is the failure mode it targets). Consumers failing drift detection are flagged as issues.

6. aggregate-mike-review.mjs checkpoint lint

packages/design-tokens/scripts/aggregate-mike-review.mjs scans all design bible markdown files (the docs/bible/ tree, walked recursively by walkMd) for [MIKE-CHECKPOINT-*) markers and reports which are pending review. This prevents a profile from shipping a color decision that's still marked as awaiting Mike's approval. A profile that references an unresolved checkpoint stops the review flow — it does not fail CI directly, but it blocks the human sign-off step.

15.3 What a profile looks like

A minimal valid profile (no overrides, consuming defaults as-is):

```
/* @zuga-technologies/design-tokens — profiles/overlay-gamer.css
 * Used by: ZugaGamerOverlay (in-game HUD)
 * Accent: master lime (no override)
 */

/* No overrides — consumes Tier 2 defaults as-is */
```

A profile with a real accent override:

```
/* @zuga-technologies/design-tokens — profiles/studio-trader.css
 * Used by: ZugaTrader
 * Accent: emerald (markets family)
 */

/* WHY: ZugaTrader uses emerald as its accent family per the markets color assign
   in the design bible §4. Emerald is distinct from master lime and communicates
   the financial/growth context of a trading product. */
--accent-brand: var(--color-emerald-700);

/* WHY: Subtle tint for trader-specific chip and badge backgrounds.
   Emerald-100 at the same lightness level as lime-100 in the master profile. */
--accent-brand-subtle: var(--color-emerald-100);

/* WHY: Strong shade used for hover/focus pressed state on accent surfaces.
   Emerald-800 maintains sufficient contrast against --accent-fg (ffffff). */
--accent-brand-strong: var(--color-emerald-800);
```

Every declaration has a WHY: comment. The lint enforces this. No WHY comment → lint failure → PR blocked.

15.4 Four-step new-profile registration gate

Step 1 — Author the profile CSS

Add <profile-slug>.css to packages/design-tokens/src/profiles/. Override only whitelisted token prefixes. Include a WHY: comment before every override declaration.

Step 2 — Register the consumer

Add a row to consuming-repos.json:

```
{
  "consumer": "ZugaTrader",
  "repo": "github.com/zuga-technologies/ZugaTrader",
  "profile": "studio-trader",
  "accentRamp": "emerald"
}
```

Also create a .zuga-design.json manifest in the consumer repo root with { "profile": "studio-trader" }.

Step 3 — Author the profile doc

Create docs/bible/profiles/<profile-slug>.md with all fields required by the Astro Zod schema (profile, consumer, family, accentRamp, rationale ≥ 40 chars, deviations). The astro check step will fail if this file is missing or malformed.

Step 4 — Open a PR and pass CI

CI runs three checks:

tokens-lint.mjs on the new profile CSS — zero violations required.

astro check — zero type errors required. This validates the profile doc schema.

drift-detect.mjs — consumer registry must be consistent.

All three must pass before merge. There is no exception path. If any check fails, fix the violation and push again.

15.5 New-component upstream flow

When a studio engineer authors a new component that's specific to their studio:

Build the component locally with --component-* tokens scoped to that component.

If the component is useful to the broader system, open a proposal in the design-tokens repo with: the component name, the tokens it introduces, an ADR stub (see §17), and at least one consuming case.

Tokens are added to the system only when ≥ 1 component actually uses them (anti-speculation rule — see §17).

Studio-specific component tokens that will never be shared stay in the studio repo and don't need a bible entry. They must still use the `--component-*` prefix and must not conflict with existing system component tokens.

This upstream flow prevents the common failure mode where studios build parallel token systems that can never be reconciled with the canonical package.

16. Cross-platform

16. Cross-platform

Authored: 2026-04-26 Authoring task: see plan Task 27

Token names are stable across all platforms. The *delivery format* differs — Phase 0 ships CSS custom properties only. Phase 2 will introduce Style Dictionary to generate platform-native formats from the same DTCG-aligned JSON source. Phase 2 is a governance roadmap item (see §17); specific version commitments and transform configs will be captured in ADRs when that work begins.

16.1 Platform mapping table

Platform	Phase	Format	Example
Web (any framework)	0 — shipped	CSS custom property	<code>var(--color-cyan-500)</code>
Electron (overlay, desktop apps)	0 — shipped	CSS custom property (identical to web)	<code>var(--color-cyan-500)</code>
Tauri WebView (Rust + web frontend)	0 — shipped	CSS custom property (WebView renders Chromium)	<code>var(--color-cyan-500)</code>
Tauri + Swift (macOS native widgets)	2 — planned	Style Dictionary swift transform	<code>Token.color.cyan.500</code>
Tauri + Kotlin (Android native)	2 — planned	Style Dictionary compose transform	<code>Token.Color.Cyan500</code>
Windows AppSDK / WinUI 3	2 — planned	Style Dictionary xaml transform	<code>{StaticResource ColorCyan500}</code>
iOS native (if needed)	2 — planned	Style Dictionary swift transform	<code>Token.color.cyan.500</code>

Why web and Electron are identical

Electron renders inside Chromium. The renderer process loads the same HTML/CSS as the web build. No translation layer needed — the same `@import "@zuga-technologies/design-tokens/tokens.css"` line works in both contexts. The overlay (ZugaGamerOverlay) already ships this way.

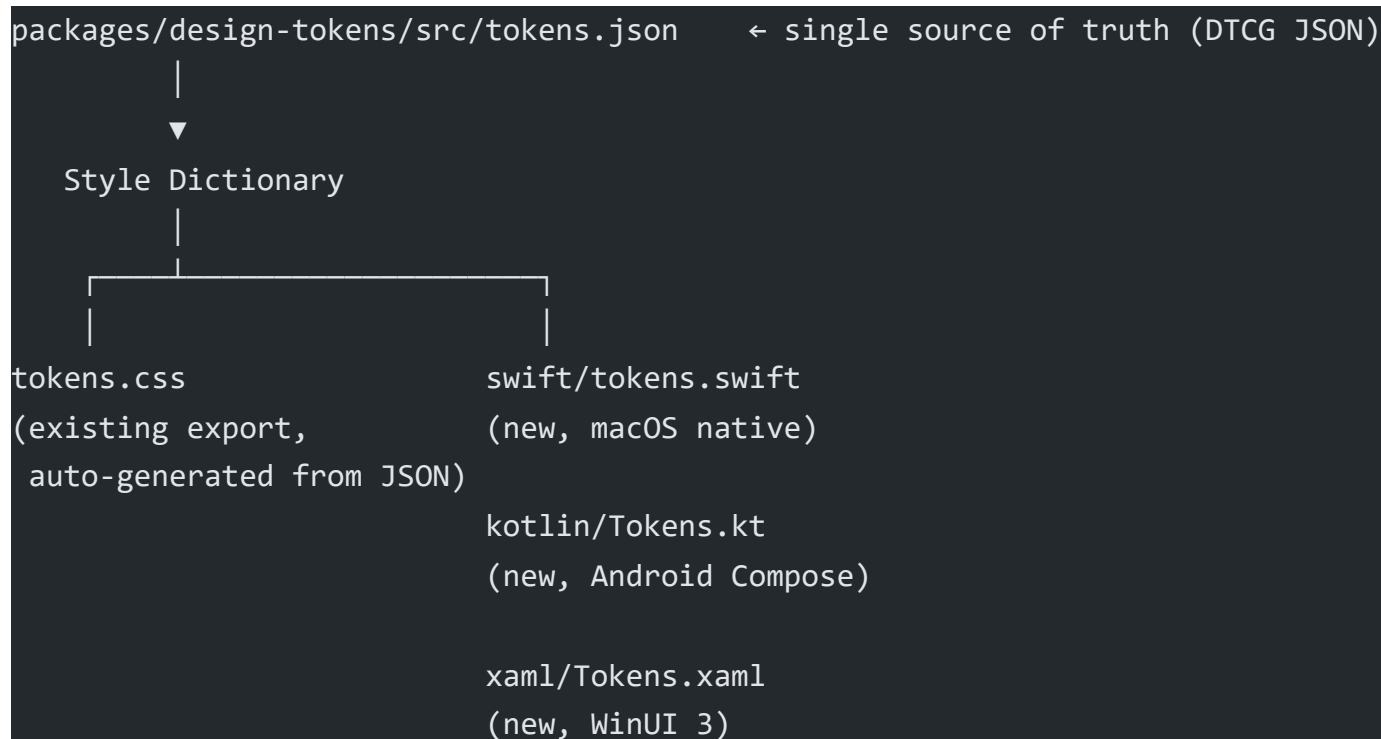
Why Tauri WebView is also identical

Tauri’s WebView wraps the OS webview engine (WKWebView on macOS, WebView2 on Windows). CSS custom properties work in all of these. The Rust side of Tauri doesn’t need to know about design tokens — it manages the window chrome, not the UI content. If a future Tauri native widget (outside the WebView) needs tokens, that’s a Phase 2 item.

16.2 Phase 2 migration path (Style Dictionary)

Phase 0 is the CSS layer only. The DTCG-aligned naming convention (`--color-{ramp}-{stop}`, `--space-{n}`, etc.) was chosen deliberately to map cleanly to the W3C Design Tokens spec and to Style Dictionary's property reference model.

When Phase 2 begins, the migration shape is:



The CSS export continues working identically for web and Electron. The new format outputs are additive. No consumer needs to change their web import.

Style Dictionary transform selection, output directory structure, and version pinning are deferred to Phase 2 ADRs. Do not pre-specify Style Dictionary versions or transform configs in this document.

16.3 Overlay considerations

ZugaGamerOverlay (Electron) and ZugaTraderOverlay (Electron) are the primary overlay consumers today. Both import `@zuga-technologies/design-tokens/tokens.css` via their profile import chain. The overlay-gamer and overlay-trader profile files live in `packages/design-tokens/src/profiles/`.

Overlays have one additional concern: performance. The full `tokens.css` is ~350 lines; this is negligible for web apps but acceptable to audit in overlay contexts where startup latency is visible. If an overlay requires a stripped token set for startup performance, that is a Phase 2 optimization — add an ADR, don't create a parallel token file.

16.4 What does not change across platforms

Token **names** are the cross-platform contract, not values. When a native platform generates `Token.color.cyan.500`, it resolves to the same hex value (`#06b6d4`) as `--color-cyan-500` in

CSS. The translation is purely syntactic. The semantic meaning, the WCAG compliance thresholds, and the RACI ownership are all identical regardless of delivery format.

17. Governance

17. Governance

Authored: 2026-04-26 Authoring task: see plan Task 27

This section owns the decision-making and change-management rules for the design system. The sub-brand contract's enforcement mechanisms are specified in §15; governance owns the human processes that wrap them.

17.1 Semver — strict definitions

The `@zuga-technologies/design-tokens` package follows semantic versioning. The definitions below are intentionally narrower than default semver convention — this is a design system, where “breaking” includes visual breaks, not just compile breaks.

MAJOR — breaking change

A major version increment is required when any of the following occur:

A token is **removed** from the public surface (`tokens.css`, any profile, any component CSS).

A token is **renamed** (renaming = remove old + add new, which is a break for consumers using the old name).

A token's **resolved value changes in a way that shifts downstream visuals detectably** — e.g., changing `--color-cyan-500` from `#06b6d4` to a different hex is a major change even if the name stays the same, because every consumer's rendered UI shifts without their consent.

A profile CSS file is **removed** from the package.

Examples: removing `--accent-brand-subtle`, renaming `--surface-canvas` to `--surface-bg`, changing `--color-slate-900` to a different value.

MINOR — additive change

A minor version increment is required when:

A new token is **added** to `tokens.css` or a component CSS file.

A new **profile** is added to `packages/design-tokens/src/profiles/`.

A new **component CSS** file is added under `components/`.

No existing consumer code breaks. Consumers opt in to new tokens by using them.

Examples: adding `--feedback-neutral` and `--feedback-neutral-subtle`, adding `profiles/studio-audio.css`.

PATCH — fix

A patch increment is required when:

A token's value is **corrected** (e.g., a hex typo that was never the intended value, documented as a bug fix in release notes).

A documentation comment inside a shipped file is corrected (typo fix, clarification).

A CI script or lint rule is fixed without changing the token surface.

17.2 Deprecation lifecycle

Removing a token requires a minimum one-calendar-quarter window between deprecation and deletion.

Step 1 — Deprecate (minor release N)

Token stays in place and continues to work.

Add a CSS comment: `/* DEPRECATED in vN.x.x – use --replacement-token instead. Will be removed in vN+1.0.0 */`.

Add an entry to `CHANGELOG.md` in the deprecation section.

Notify all registered consumers via the `consuming-repos.json` registry (manual announcement or automated GH issue).

Step 2 — Confirm no active consumers (12-week minimum window)

Run `drift-detect.mjs` to verify no registered consumer is still referencing the deprecated token. If consumers remain, extend the window. Do not remove until all registered consumers have migrated.

Step 3 — Remove (major release N+1)

Delete the token from `tokens.css` (or the relevant file).

Bump to major version.

Add entry to `CHANGELOG.md` in the breaking-changes section.

The 12-week minimum is a floor, not a target. If the system is early-stage and consumers can migrate quickly, you can compress the window — but document the compressed timeline in the release notes and get explicit confirmation from all affected consumers before proceeding.

17.3 ADR format

Architecture Decision Records live at `docs/bible/adr/`. T29 will populate the first set of ADRs.

Every ADR uses this structure:

```
---
id: <integer>
title: "<short decision title>"
date: "YYYY-MM-DD"
status: proposed | accepted | deprecated | superseded
context: "<one paragraph – what situation required a decision>"
decision: "<what was decided>"
consequences: "<what changes, what becomes easier, what becomes harder>"
supersededBy: <id> # optional – only if status is superseded
---
```

ADRs are write-once. Once accepted, an ADR is never edited to change the decision — if the decision changes, write a new ADR that supersedes it and update the old ADR's status to

superseded with a supersededBy pointer.

New token categories (not just new values within an existing ramp) require an ADR. See §17.4.

17.4 Token-add policy — anti-speculation rule

Tokens are added to the system when **≥ 1 component or profile actually uses them**. “We might need this later” is not a justification. Unused tokens create surface area that must be versioned, documented, and eventually deprecated. They also mislead engineers about what’s available.

Additional rules:

A new value in an existing ramp (e.g., adding --color-cyan-925) requires a MINOR version increment but does **not** require an ADR.

A **new category** (e.g., adding a --border-radius-* semantic alias layer that doesn’t currently exist) requires an ADR before the token is added. This prevents system architecture from drifting through accumulation.

A new profile for a studio requires both a consuming-repos.json entry and a profile doc (docs/bible/profiles/<slug>.md) before the CSS is accepted. See §15 for the four-step gate.

17.5 RACI

RACI for design system decisions. This reflects the actual division of labor, not a template.

Decision type	Antonio	Mike	Studio engineer	Other studios
Master accent hue (cyan family assignment)	A	R (veto)	—	I
Color ramp values (Tier 1 hex values)	R / A	C	—	I
Semantic token assignments (which ramp stop maps to which role)	R / A	C	—	I
New token category (requires ADR)	R / A	C	C	I
New value in existing ramp (minor add)	R / A	I	—	I
Studio accent ramp assignment	A	R (approval)	C	I
Per-studio component token overrides	—	I	R / A	I
Breaking change (major version)	R / A	C	—	I
Deprecation announcement	R / A	I	C	I
Profile registration gate (new profile)	A	—	R	I
ADR status changes	R / A	C	—	I

Key:

- R** — Responsible (does the work)
- A** — Accountable (decision stops here; only one per row)
- C** — Consulted (must be heard before decision)
- I** — Informed (notified of outcome)

Mike's veto applies to brand decisions in his lanes (wordmark adjacency, hue assignments, voice in external-facing copy). Antonio holds accountability for shipped visual decisions across the system. Studio engineers own their integration work and are responsible for passing CI before requesting merge.

17.6 Quarterly drift audit cadence

`packages/design-tokens/scripts/drift-detect.mjs` runs nightly (GH Actions cron, T19). This catches mechanical drift automatically.

The **quarterly audit** (every 12 weeks) is the human-in-the-loop sweep:

Antonio reviews the accumulated drift-detect output for the quarter.

Any consumer flagged for drift (outdated version, non-registry token import) receives a filed GitHub issue with a migration target date.

Consumers that have been on a deprecated token for longer than the 12-week window get escalated — their profile is reviewed for forced migration in the next major release cycle.

Quarterly audit results are recorded as a comment in the relevant ADR or as a new short ADR if the audit reveals a systemic issue.

The quarterly cadence aligns with the deprecation lifecycle window, so a token deprecated at the start of Q1 is eligible for removal at the Q2 audit if all consumers have migrated. This creates a predictable rhythm for consumers to plan their upgrade work.

Phase 2 platform expansion (Style Dictionary, native formats) is a governance roadmap item for the quarterly review when native consumer repos come online. See §16 for the migration shape.

18. Glossary

18. Glossary

Authored: 2026-04-26 Authoring task: see plan Task 27

Terminology used consistently across the design bible, codebase, and internal communications. When a term has a user-facing restriction (what NOT to call it in product copy), that is noted explicitly.

Product names

These three are the highest-confusion cluster because they overlap in context. Keep them distinct.

Zugabot

The autonomous agent — the cognitive core of the Zuga system. Runs as a daemon on the Mac Mini. Responsible for autonomous task execution, memory, tool use, and the agent loop. Zugabot is an *experiment* / infrastructure layer, not a user-facing product name in isolation. In user-facing copy, refer to specific capabilities by studio name, not “Zugabot.”

See: docs/bible/01-mission.md for mission context.

ZugaChat

The user-facing chat surface. The place where users interact with Zugabot’s conversational layer. ZugaChat is distinct from Zugabot itself — ZugaChat is the interface, Zugabot is the engine. In product copy, say “ZugaChat” when referring to the chat feature, not “Zugabot.”

ZugaLife

The wellness studio. One of ~21 studios within ZugaApp. Focuses on habits, meditation, fitness, and health tracking. The accent ramp for ZugaLife is the mint/teal family (`--color-mint-*`). ZugaLife is a product name — users see it. The billing currency inside ZugaLife uses ZugaTokens (see below).

Architecture terms

Branded House

The brand architecture pattern Zuga uses. A single master brand (Zuga) with sub-brands that share the prefix (ZugaLife, ZugaTrader, ZugaCode, ZugaForge, etc.). The opposite of a “house of brands” (where each product has a fully independent brand identity). The Branded House pattern means every studio inherits the master accent color family and the Zuga prefix before introducing its own personality. See §2 for the full brand architecture.

Drift

When a consumer’s token usage diverges from what the registry says it should be. Drift has two forms:

Version drift — consumer is using an outdated @zuga-technologies/design-tokens version.

Usage drift — consumer is importing or defining tokens outside the @zuga-technologies/design-tokens package (e.g., copy-pasting tokens.css into their repo).

`drift-detect.mjs` catches both automatically on a nightly schedule. The quarterly audit is the human-in-the-loop check. See §17.6.

Food court

The host shell — ZugaApp. The place studios (food trucks) live. This is an internal metaphor used in development conversations to describe the relationship between the ZugaApp shell and its studio units. **Do not use “food court” in user-facing copy.** Users see “ZugaApp.”

Food truck

A studio. The integrated unit that plugs into ZugaApp. Each studio is self-contained (its own backend, its own plugin, its own profile). **Do not use “food truck” in user-facing copy.** Users see the studio’s product name (ZugaLife, ZugaCode, etc.).

Master accent

The master Zuga accent hue: `--color-lime-400` (hex `#a3e635`) from the lime ramp. The semantic token that carries it in production is `--accent-brand`, which resolves to `--color-lime-400` paired with a DARK foreground `--accent-fg: #0a0a0a`. Lime is light-forward, so — unlike the former cyan master (ADR-0002) — the identity stop is also the functional stop: a filled lime element with dark text clears WCAG AAA on both light and dark surfaces. For accent TEXT on a light surface, use lime-700 (`#4d7c0f`). “Master accent” = lime family (unified 2026-06-30, ADR-0004; cyan was the master until then and is now the feedback-info hue). Studio sub-brands use different ramps but are all sub-ordinate to this master identity.

Mike checkpoint

A `[MIKE-CHECKPOINT-*)` marker in source files or design docs. Marks a decision that requires Mike’s review before it can ship. Unresolved checkpoints appear in the output of `aggregate-mike-review.mjs`. Once approved by Mike, the marker flips to `[MIKE-APPROVED-*)`. There are 45 checkpoints in the design bible as of T26.

Mike-approved

The resolved form of a Mike checkpoint. `[MIKE-APPROVED-*)` indicates the decision at that marker has received Mike’s explicit approval and can proceed to production. See §17.5 for Mike’s RACI role.

OpenClaw

A third-party Manus-derived agent framework. **Not owned by Zuga.** The naming distinction matters: OpenClaw is the external tool Zuga integrates with; ZugaClaw is Zuga’s own wrapper and tooling around it. Never conflate these in code, comments, or documentation. Also: never put Claude API tokens into OpenClaw — this risks account suspension. See memory rule `feedback_openclaw_subscription_ban.md`.

Profile

A sub-brand override CSS file. Lives at `packages/design-tokens/src/profiles/<slug>.css`. Contains only the token overrides a studio is allowed to make (accent, density, component). Must

be registered in `consuming-repos.json` and documented in `docs/bible/profiles/<slug>.md`. Enforced by `tokens-lint.mjs`. See §15 for the full sub-brand contract.

Studios

The plug-in app units inside ZugaApp. Each studio is a self-contained product experience: its own backend, its own Vue plugin, its own profile. The ~21 active studios include ZugaLife, ZugaTrader, ZugaCode, Zugalmage, ZugaCloud, ZugaAudio, ZugaForge, and others. In user-facing copy, studios are called by their product name — never “apps,” never “plugins,” never “modules.” Internally, “studio” and “food truck” are interchangeable.

Tier 1 / Tier 2 / Tier 3 tokens

The three-layer token architecture:

Tier 1 — Primitives. Raw values. Brand-agnostic. Color ramps, spacing scale, type scale, radii, shadows, motion durations, z-index levels, containers, breakpoints. Locked — no profile can redefine them. Prefix: `--color-*`, `--space-*`, `--radius-*`, `--shadow-*`, `--motion-*`, `--z-*`, `--container-*`, `--breakpoint-*`, `--type-*`, `--font-*`.

Tier 2 — Semantic. Carry meaning. Resolved from Tier 1. This is what components use. Divided into locked (`--surface-*`, `--text-*`, `--border-*`, `--feedback-*`) and overridable (`--accent-*`, `--density-*`). Profiles may only override the overridable Tier 2 group.

Tier 3 — Component. Scoped to a single component. Declared in component CSS files under `packages/design-tokens/src/components/`. Overridable per profile. Prefix: `--component-{name}-{property}`.

See §14 for the full naming convention and code examples.

ZugaClaw

Zuga’s in-house wrapper and tooling around the OpenClaw agent framework. ZugaClaw is owned by Zuga Technologies. This is the distinction that matters in all code references and documentation. When you see ZugaClaw, it refers to Zuga’s code. When you see OpenClaw, it refers to the third-party upstream.

ZugaTokens

The user credit currency. Always written with capital Z and T. Used across all studios for premium features, API consumption, and marketplace transactions. In user-facing copy, always say “ZugaTokens” — never “credits,” never “tokens” (lowercase, generic). The token economy is managed at the ZugaApp level, not per-studio.